

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN CUỐI KỲ
MÔN HỌC: CÁC HỆ THỐNG PHÂN TÁN
ĐỀ TÀI: PHÁT TRIỂN HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ
THEO KIẾN TRÚC MICROSERVICE

Giảng viên hướng dẫn: Kim Ngọc Bách

Thành viên nhóm

Họ và tên	Mã sinh viên
Nguyễn Đức Toàn	B18DCCN540
Bùi Việt Anh	B21DCCN138
Cù Xuân Hòa	B21DCCN375

Hà Nội – 2026

LỜI CẢM ƠN

Lời đầu tiên, nhóm chúng em xin được gửi lời cảm ơn chân thành và sâu sắc nhất tới các thầy cô giáo của Học viện đã tận tình giảng dạy, truyền đạt những kiến thức và kinh nghiệm quý báu trong suốt quá trình học tập. Những kiến thức đó là nền tảng quan trọng giúp chúng em thực hiện đề án cuối kì môn của Các hệ thống phân tán này.

Nhóm chúng em xin gửi lời cảm ơn sâu sắc đến thầy Kim Ngọc Bách, người đã tận tình hướng dẫn, chỉ bảo và hỗ trợ chúng em trong suốt quá trình thực hiện bài tập lớn. Những góp ý và định hướng của thầy là cơ sở quan trọng giúp nhóm hoàn thành đề tài.

Với những kiến thức được học cùng với sự hướng dẫn của thầy, nhóm chúng em đã cố gắng hoàn thành đề án cuối kì này. Tuy nhiên, do hạn chế về thời gian và kinh nghiệm, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ thầy để có thể hoàn thiện hơn.

Nhóm chúng em xin trân trọng cảm ơn gia đình và người thân đã luôn đồng hành, chia sẻ những khó khăn và tạo nền tảng vững chắc cả về tinh thần lẫn vật chất trong suốt quá trình chúng em thực hiện báo cáo đề án cuối kì.

BẢNG ĐÁNH GIÁ MỨC ĐỘ THAM GIA CỦA THÀNH VIÊN NHÓM

STT	Tên thành viên	Mã sinh viên	Vai trò	Mức độ tham gia
1	Nguyễn Đức Toàn	B18DCCN540	Trưởng nhóm	
2	Bùi Việt Anh	B21DCCN318	Thành viên	
3	Cù Xuân Hòa	B21DCCN375	Thành viên	

MỤC LỤC

LỜI CẢM ƠN	1
BẢNG ĐÁNH GIÁ MỨC ĐỘ THAM GIA CỦA THÀNH VIÊN NHÓM.....	2
MỤC LỤC	3
DANH SÁCH CÁC TỪ VIẾT TẮT	5
DANH SÁCH HÌNH VẼ	7
DANH SÁCH BẢNG BIỂU	8
LỜI MỞ ĐẦU	9
CHƯƠNG I: CƠ SỞ LÝ THUYẾT	12
1.1. Khái niệm hệ thống phân tán.....	12
1.2. Tính trong suốt, khả năng mở rộng và khả năng chịu lỗi	13
1.3. Kiến trúc microservices và vai trò của API Gateway	14
1.4. Mô hình database-per-service	15
1.5. Giao tiếp REST đồng bộ và hạn chế của mô hình gọi liên dịch vụ theo chuỗi ..	16
1.6. RabbitMQ, exchange, queue, routing key và mô hình publish/subscribe	17
1.7. Dead-letter queue và cơ chế retry	19
1.8. Kiến trúc hướng sự kiện và tính nhất quán cuối cùng	20
1.9. Tracing và observability trong hệ thống phân tán.....	22
1.10. Docker Compose trong triển khai hệ thống nhiều dịch vụ	23
Kết luận chương	24
CHƯƠNG 2: PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ HỆ THỐNG.....	25
2.1. Nhận diện bài toán và phạm vi hệ thống.....	25
2.2. Kiến trúc tổng thể của hệ thống	26
2.3. Các sơ đồ tuần tự của hệ thống	31
2.4. Thiết kế giao tiếp bất đồng bộ qua RabbitMQ.....	36
2.5. Thiết kế dữ liệu theo mô hình database-per-service	38
Kết luận chương	40
CHƯƠNG 3: TRIỂN KHAI VÀ KẾT QUẢ	41
3.1. Cấu trúc triển khai của dự án	41
3.2. Kết quả triển khai giao diện đăng nhập và đăng ký	43
3.3. Kết quả triển khai giao diện danh sách sản phẩm	46
3.4. Kết quả triển khai giao diện giỏ hàng	47

3.5. Kết quả triển khai luồng tạo đơn hàng	49
3.6. Kết quả triển khai giao diện quản trị sản phẩm.....	50
3.7. Đánh giá kết quả đạt được.....	51
3.8. Hạn chế của hệ thống	52
3.9. Hướng cải tiến và phát triển.....	52
Kết luận chương	53
TỔNG KẾT	54
TÀI LIỆU THAM KHẢO	56

DANH SÁCH CÁC TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ	Giải nghĩa
API	Application Programming Interface	Giao diện lập trình dịch vụ
BFF	Backend for Frontend	Biến thể của API Gateway
CB	Circuit Breaker	Cơ chế ngắt mạch chống lỗi dây chuyền
CORS	Cross-Origin Resource Sharing	Kiểm soát truy cập khác nguồn
DLQ	Dead Letter Queue	Hàng đợi chứa message lỗi sau khi retry thất bại
EDA	Event-Driven Architecture	Kiến trúc hướng sự kiện
HTTP	Hypertext Transfer Protocol	Giao thức REST đồng bộ giữa client và service
JPA	Java Persistence API	Truy cập dữ liệu trong các service Spring Boot
MDC	Mapped Diagnostic Context	Gắn traceId, spanId vào log
N+1 Call	Anti-pattern gọi lặp sang service khác	Xuất hiện trong luồng checkout
REST	Representational State Transfer	Phong cách API đồng bộ
RPO	Recovery Point Objective	Chỉ số tham chiếu cho dữ liệu sau lỗi
RTO	Recovery Time Objective	Chỉ số tham chiếu cho thời gian phục hồi
SLO	Service Level Objective	Mục tiêu vận hành đề xuất
SPOF	Single Point of Failure	Điểm lỗi đơn

TTL	Time To Live	Thuộc tính hàng đợi/message nếu mở rộng
UI	User Interface	Frontend cho người dùng và admin
YAML	YAML Ain't Markup Language	Định dạng cấu hình application.yml, Compose

DANH SÁCH HÌNH ẢNH

Hình 1.1. API Gateway trong kiến trúc microservices [1].....	11
Hình 1.2. Database-per-service trong microservices [5].....	16
Hình 1.3. Minh họa RabbitMQ exchange, queue, routing key và DLQ [8].....	20
Hình 2.1. Sơ đồ tổng quan của hệ thống thương mại điện tử theo kiến trúc Microservices.....	28
Hình 2.2. Các công nghệ chính được sử dụng trong hệ thống.....	29
Hình 2.3. Sơ đồ tuần tự cho luồng đăng ký người dùng.....	31
Hình 2.4. Sơ đồ tuần tự cho luồng đăng nhập người dùng.....	32
Hình 2.5. Sơ đồ tuần tự cho luồng duyệt và tìm kiếm sản phẩm.....	33
Hình 2.6. Sơ đồ tuần tự cho luồng thêm sản phẩm vào giỏ hàng.....	34
Hình 2.7. Sơ đồ tuần tự cho luồng checkout và tạo đơn hàng.....	35
Hình 2.8. Sơ đồ luồng sự kiện RabbitMQ sau khi tạo đơn hàng.....	37
Hình 2.9. Sơ đồ database-per-service của hệ thống.....	39
Hình 3.1. Cây thư mục của dự án.....	43
Hình 3.2. Giao diện đăng nhập của hệ thống.....	45
Hình 3.3. Giao diện đăng ký của hệ thống.....	45
Hình 3.4. Giao diện danh sách sản phẩm của hệ thống.....	46
Hình 3.5. Giao diện giỏ hàng của hệ thống.....	48
Hình 3.6. Giao diện đặt đơn hàng thành công của hệ thống.....	49
Hình 3.7. Giao diện admin quản trị sản phẩm của hệ thống.....	50

DANH SÁCH BẢNG BIỂU

Bảng 2.1. Các thành phần và cơ sở dữ liệu của hệ thống.....	29
---	----

LỜI MỞ ĐẦU

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, các hệ thống phần mềm ngày càng phải phục vụ số lượng người dùng lớn hơn, xử lý nhiều loại nghiệp vụ phức tạp hơn và đảm bảo khả năng hoạt động ổn định trong nhiều điều kiện khác nhau. Đặc biệt đối với các hệ thống thương mại điện tử, yêu cầu đặt ra không chỉ dừng lại ở việc hiển thị sản phẩm và tiếp nhận đơn hàng, mà còn bao gồm quản lý người dùng, quản lý giỏ hàng, xử lý đặt hàng, cập nhật tồn kho, theo dõi lịch sử giao dịch và hỗ trợ quản trị hệ thống. Khi quy mô chức năng tăng lên, việc thiết kế hệ thống theo mô hình nguyên khối truyền thống thường bộc lộ nhiều hạn chế về khả năng bảo trì, mở rộng, triển khai và xử lý lỗi.

Trong mô hình nguyên khối, toàn bộ nghiệp vụ thường được xây dựng trong một ứng dụng duy nhất. Cách tiếp cận này có ưu điểm là dễ triển khai ở giai đoạn đầu, cấu trúc đơn giản và phù hợp với các hệ thống nhỏ. Tuy nhiên, khi hệ thống phát triển, mã nguồn ngày càng lớn, các module phụ thuộc chặt chẽ vào nhau, việc thay đổi một chức năng có thể ảnh hưởng đến nhiều phần khác. Bên cạnh đó, toàn bộ ứng dụng thường phải được build và triển khai lại cùng lúc, gây khó khăn cho việc phát triển độc lập giữa các nhóm chức năng. Nếu một thành phần bên trong hệ thống gặp lỗi nghiêm trọng, toàn bộ ứng dụng có thể bị ảnh hưởng, làm giảm tính sẵn sàng của hệ thống.

Để khắc phục những hạn chế trên, kiến trúc microservices được sử dụng như một hướng tiếp cận phổ biến trong thiết kế hệ thống phần mềm hiện đại. Thay vì xây dựng toàn bộ nghiệp vụ trong một khối duy nhất, microservices chia hệ thống thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một miền nghiệp vụ riêng biệt. Các dịch vụ có thể được phát triển, triển khai và mở rộng độc lập, đồng thời giao tiếp với nhau thông qua các giao thức mạng như HTTP/REST hoặc thông qua cơ chế truyền thông bất đồng bộ bằng message broker. Cách tổ chức này giúp hệ thống linh hoạt hơn, dễ bảo trì hơn và phù hợp hơn với các bài toán có nhiều thành phần cần phối hợp trong môi trường phân tán.

Đối với một hệ thống thương mại điện tử, việc áp dụng kiến trúc microservices mang lại nhiều lợi ích rõ rệt. Chẳng hạn, chức năng quản lý người dùng có thể được tách thành một dịch vụ riêng, chức năng quản lý sản phẩm được xử lý bởi một dịch vụ khác, giỏ hàng do một dịch vụ độc lập đảm nhiệm, trong khi nghiệp vụ đặt hàng được tổ chức thành một dịch vụ trung tâm để điều phối quá trình mua hàng. Khi đó, mỗi dịch vụ có thể sở hữu cơ sở dữ liệu riêng, phù hợp với nguyên tắc database-per-service. Cách thiết kế này giúp giảm sự phụ thuộc trực tiếp giữa các miền nghiệp vụ, đồng thời tạo điều kiện thuận lợi cho việc mở rộng từng thành phần theo nhu cầu thực tế.

Tuy nhiên, kiến trúc microservices cũng đặt ra nhiều thách thức mới. Khi hệ thống được chia thành nhiều dịch vụ, các lời gọi trong cùng một tiến trình được thay thế bằng giao tiếp qua mạng. Điều này làm phát sinh các vấn đề đặc trưng của hệ thống phân tán như độ trễ truyền thông, lỗi kết nối, dịch vụ phụ thuộc tạm thời không khả dụng, dữ liệu phân tán ở nhiều cơ sở dữ liệu khác nhau và khó đảm bảo tính nhất quán

tức thời trên toàn hệ thống. Vì vậy, bên cạnh việc tách dịch vụ, hệ thống cần có các cơ chế hỗ trợ như API Gateway để cung cấp điểm truy cập thống nhất cho phía người dùng, message broker để xử lý các tác vụ bất đồng bộ, circuit breaker để hạn chế lỗi dây chuyền, retry để thử lại khi xảy ra lỗi tạm thời, dead-letter queue để lưu các thông điệp xử lý thất bại, và cơ chế truy vết để theo dõi luồng xử lý đi qua nhiều thành phần.

Trong đề tài này, nhóm tập trung nghiên cứu, phân tích và triển khai một hệ thống thương mại điện tử phân tán theo kiến trúc microservices. Hệ thống được tổ chức thành nhiều thành phần độc lập, bao gồm giao diện người dùng, API Gateway, các dịch vụ nghiệp vụ như quản lý người dùng, quản lý sản phẩm, quản lý giỏ hàng và xử lý đơn hàng. Bên cạnh đó, hệ thống sử dụng cơ sở dữ liệu riêng cho từng dịch vụ nhằm đảm bảo tính độc lập về dữ liệu, đồng thời sử dụng message broker để truyền tải sự kiện sau khi đơn hàng được tạo. Cách tiếp cận này cho phép hệ thống kết hợp cả giao tiếp đồng bộ và bất đồng bộ, qua đó phản ánh rõ các đặc trưng quan trọng của một hệ thống phân tán.

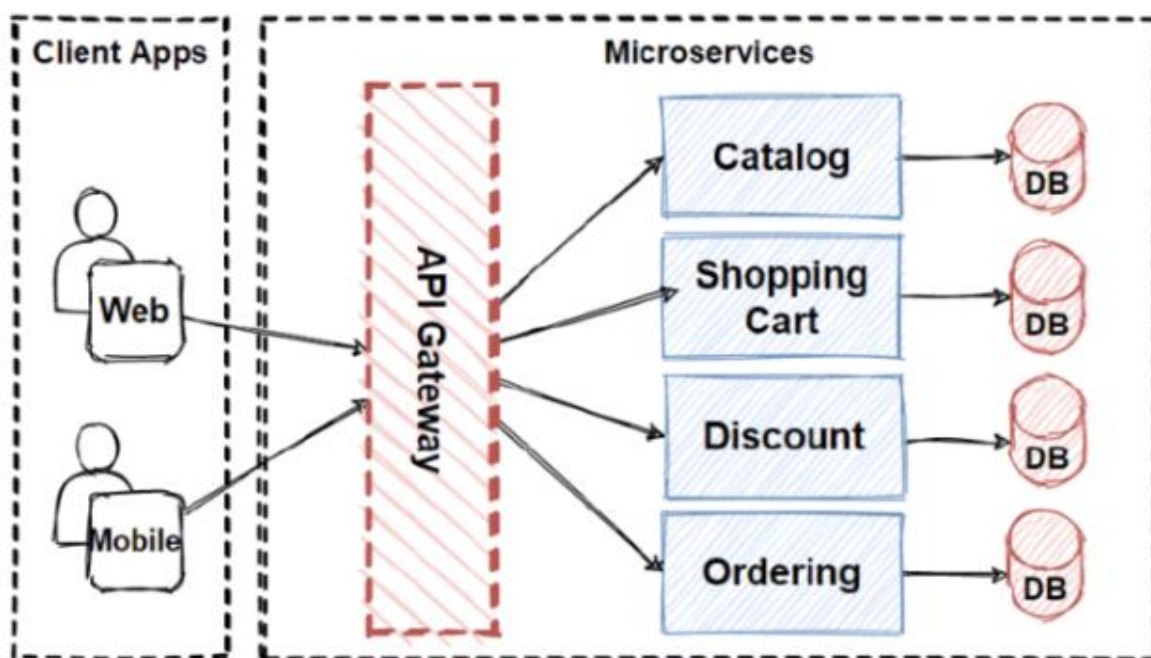
Luồng nghiệp vụ đặt hàng là một trong những nội dung trọng tâm của hệ thống. Khi người dùng thực hiện thao tác đặt hàng, dịch vụ xử lý đơn hàng cần lấy thông tin giỏ hàng, kiểm tra thông tin sản phẩm, tính tổng giá trị đơn hàng, lưu thông tin đơn hàng và phát sinh sự kiện để các dịch vụ khác tiếp tục xử lý. Sau khi đơn hàng được tạo, các tác vụ như giảm số lượng tồn kho, cập nhật thông tin người dùng hoặc xóa giỏ hàng có thể được thực hiện thông qua cơ chế bất đồng bộ. Điều này giúp giảm mức độ phụ thuộc trực tiếp giữa các dịch vụ, đồng thời thể hiện rõ nguyên lý nhất quán cuối cùng trong hệ thống phân tán.

Việc triển khai hệ thống cũng gắn liền với các vấn đề quan trọng của môn học Các hệ thống phân tán. Thứ nhất, hệ thống thể hiện tính phân tán thông qua việc nhiều dịch vụ độc lập cùng phối hợp để cung cấp một chức năng thống nhất cho người dùng. Thứ hai, hệ thống sử dụng cơ chế truyền thông qua mạng, trong đó có cả lời gọi REST đồng bộ và trao đổi thông điệp bất đồng bộ. Thứ ba, dữ liệu được phân tách theo từng dịch vụ, dẫn đến yêu cầu phải xử lý tính nhất quán dữ liệu giữa các thành phần. Thứ tư, hệ thống cần có khả năng chịu lỗi ở mức nhất định khi một dịch vụ phụ thuộc bị lỗi hoặc khi việc xử lý thông điệp không thành công. Những vấn đề này đều là các nội dung cốt lõi khi nghiên cứu và xây dựng hệ thống phân tán trong thực tế.

Bên cạnh các ưu điểm, đề tài cũng đặt ra yêu cầu phải đánh giá một cách khách quan những hạn chế còn tồn tại của hệ thống. Một hệ thống microservices nếu chỉ tách thành nhiều dịch vụ mà thiếu bảo mật, thiếu cơ chế giám sát tập trung, chưa có chiến lược xử lý giao dịch phân tán rõ ràng hoặc chưa đảm bảo tính lũy đẳng khi xử lý thông điệp thì vẫn tiềm ẩn nhiều rủi ro trong vận hành. Do đó, báo cáo không chỉ trình bày thiết kế và triển khai hệ thống, mà còn phân tích các điểm mạnh, điểm yếu, rủi ro kỹ thuật và hướng cải tiến trong tương lai. Các nội dung như xác thực người dùng, phân quyền, mã hóa mật khẩu, quản lý cấu hình, lưu trữ bền vững, tối ưu hiệu năng, kiểm thử chịu tải và nâng cao khả năng quan sát hệ thống đều được xem xét như các hướng phát triển cần thiết.

Từ những lý do trên, đề tài “Phát triển hệ thống thương mại điện tử theo cấu trúc Microservices” được lựa chọn nhằm vận dụng các kiến thức của môn Các hệ thống phân tán vào một bài toán phần mềm cụ thể. Thông qua đề tài, nhóm có cơ hội tìm hiểu cách tổ chức một hệ thống nhiều dịch vụ, cách các thành phần giao tiếp với nhau, cách xử lý dữ liệu phân tán, cách xây dựng luồng nghiệp vụ bất đồng bộ và cách đánh giá hệ thống dưới góc nhìn mở rộng, chịu lỗi và bảo mật. Đây là cơ sở quan trọng giúp củng cố kiến thức lý thuyết và rèn luyện khả năng phân tích, thiết kế, triển khai hệ thống phần mềm trong môi trường thực tế.

Báo cáo được tổ chức thành ba chương chính. Chương 1 trình bày cơ sở lý thuyết liên quan đến hệ thống phân tán, kiến trúc microservices, API Gateway, giao tiếp REST, message broker, database-per-service, cơ chế nhất quán cuối cùng, circuit breaker, retry, dead-letter queue và Docker Compose. Chương 2 tập trung phân tích bài toán và thiết kế hệ thống, bao gồm kiến trúc tổng thể, các thành phần chức năng, luồng dữ liệu, API, cơ sở dữ liệu và cơ chế truyền thông giữa các dịch vụ. Chương 3 trình bày quá trình triển khai, các kịch bản thử nghiệm, đánh giá hệ thống, phân tích bảo mật, những hạn chế còn tồn tại và đề xuất hướng phát triển. Cuối cùng, phần kết luận tổng hợp những kết quả đạt được, đồng thời nêu ra các định hướng cải tiến nhằm hoàn thiện hệ thống trong tương lai.



Hình 1.1: API Gateway trong kiến trúc microservices [1].

CHƯƠNG I: CƠ SỞ LÝ THUYẾT

Chương này trình bày các cơ sở lý thuyết quan trọng liên quan đến việc xây dựng một hệ thống thương mại điện tử theo kiến trúc phân tán. Nội dung của chương tập trung vào những khái niệm nền tảng như hệ thống phân tán, tính trong suốt, khả năng mở rộng, khả năng chịu lỗi, kiến trúc microservices, API Gateway và mô hình database-per-service. Đây là những kiến thức cần thiết để làm rõ cách các thành phần trong hệ thống phối hợp với nhau, cách dữ liệu được tổ chức, cũng như cách hệ thống xử lý các vấn đề đặc trưng của môi trường phân tán.

Trong các hệ thống phần mềm hiện đại, đặc biệt là các hệ thống thương mại điện tử, yêu cầu đặt ra không chỉ là xây dựng được các chức năng nghiệp vụ riêng lẻ, mà còn phải đảm bảo khả năng vận hành ổn định, mở rộng linh hoạt và giảm thiểu ảnh hưởng khi một thành phần gặp sự cố. Khi hệ thống có nhiều chức năng như quản lý người dùng, quản lý sản phẩm, quản lý giỏ hàng, xử lý đơn hàng và cập nhật tồn kho, việc tổ chức toàn bộ nghiệp vụ trong một ứng dụng nguyên khối có thể gây khó khăn cho quá trình phát triển và bảo trì. Vì vậy, việc tìm hiểu các mô hình kiến trúc phân tán là cơ sở quan trọng để thiết kế hệ thống theo hướng linh hoạt và phù hợp hơn với nhu cầu thực tế.

1.1. Khái niệm hệ thống phân tán

Hệ thống phân tán là một tập hợp các máy tính, tiến trình hoặc dịch vụ độc lập, được kết nối với nhau thông qua mạng và phối hợp hoạt động nhằm cung cấp một chức năng thống nhất cho người dùng. Điểm quan trọng của hệ thống phân tán là các thành phần bên trong có thể chạy trên nhiều tiến trình, nhiều máy chủ hoặc nhiều môi trường khác nhau, nhưng đối với người dùng cuối, hệ thống vẫn được cảm nhận như một ứng dụng duy nhất.

Trong một hệ thống phần mềm thương mại điện tử, tính phân tán có thể được thể hiện thông qua việc tách các chức năng nghiệp vụ thành nhiều dịch vụ độc lập. Mỗi dịch vụ đảm nhiệm một vai trò riêng, chẳng hạn dịch vụ người dùng xử lý đăng ký và đăng nhập, dịch vụ sản phẩm quản lý danh mục hàng hóa, dịch vụ giỏ hàng lưu trạng thái mua sắm tạm thời, còn dịch vụ đơn hàng chịu trách nhiệm xử lý quá trình đặt hàng. Các dịch vụ này không hoạt động tách rời hoàn toàn, mà phối hợp với nhau thông qua giao tiếp mạng để hoàn thành một quy trình nghiệp vụ chung.

So với hệ thống nguyên khối, hệ thống phân tán có nhiều ưu điểm. Trước hết, hệ thống có thể được mở rộng theo từng thành phần thay vì phải mở rộng toàn bộ ứng dụng. Nếu một chức năng có tải cao hơn, chẳng hạn chức năng xem sản phẩm hoặc tạo đơn hàng, thành phần tương ứng có thể được ưu tiên tối ưu hoặc nhân bản. Thứ hai, việc tách thành nhiều dịch vụ giúp quá trình phát triển và bảo trì trở nên linh hoạt hơn, vì mỗi dịch vụ có phạm vi trách nhiệm rõ ràng. Thứ ba, hệ thống có thể tăng khả năng chịu lỗi, bởi sự cố ở một thành phần không nhất thiết làm toàn bộ hệ thống ngừng hoạt động ngay lập tức.

Tuy nhiên, hệ thống phân tán cũng làm phát sinh nhiều thách thức mới. Khi các thành phần giao tiếp qua mạng, hệ thống phải đối mặt với độ trễ truyền thông, lỗi kết nối, mất gói tin, timeout hoặc tình huống một dịch vụ phụ thuộc tạm thời không phản hồi. Ngoài ra, do dữ liệu có thể được lưu trữ phân tán ở nhiều cơ sở dữ liệu khác nhau, việc đảm bảo tính nhất quán tức thời trên toàn hệ thống trở nên khó khăn hơn. Vì vậy, khi thiết kế hệ thống phân tán, cần đồng thời xem xét các yếu tố như giao tiếp liên dịch vụ, đồng bộ dữ liệu, xử lý lỗi, giám sát hoạt động và bảo mật.

1.2. Tính trong suốt, khả năng mở rộng và khả năng chịu lỗi

Một đặc trưng quan trọng của hệ thống phân tán là tính trong suốt. Tính trong suốt có nghĩa là hệ thống che giấu một phần sự phức tạp bên trong khỏi người dùng hoặc khỏi các thành phần khác. Người dùng không cần biết hệ thống được cấu thành từ bao nhiêu dịch vụ, mỗi dịch vụ chạy ở đâu, dữ liệu được lưu trong cơ sở dữ liệu nào hoặc các dịch vụ giao tiếp với nhau ra sao. Thay vào đó, người dùng chỉ tương tác với một giao diện hoặc một điểm truy cập thống nhất.

Trong kiến trúc microservices, tính trong suốt thường được hỗ trợ bởi API Gateway. API Gateway đóng vai trò là điểm vào chung của hệ thống, nhận yêu cầu từ phía client và định tuyến yêu cầu đó đến dịch vụ phù hợp ở phía sau. Nhờ vậy, phía client không cần gọi trực tiếp từng dịch vụ nghiệp vụ. Cách tổ chức này giúp giảm sự phụ thuộc giữa client và cấu trúc nội bộ của hệ thống, đồng thời tạo điều kiện thuận lợi cho việc thay đổi, mở rộng hoặc tái cấu trúc các dịch vụ phía sau mà ít ảnh hưởng đến giao diện bên ngoài. [2]

Khả năng mở rộng cũng là một yêu cầu quan trọng trong hệ thống phân tán. Một hệ thống được xem là có khả năng mở rộng tốt nếu nó có thể đáp ứng khối lượng người dùng, dữ liệu hoặc giao dịch tăng lên mà không cần thay đổi toàn bộ kiến trúc. Trong mô hình microservices, khả năng mở rộng thường được thực hiện theo chiều ngang bằng cách tăng số lượng đối tượng được tạo lên dựa trên khuôn mẫu của một lớp của một dịch vụ cụ thể. Ví dụ, nếu dịch vụ xử lý đơn hàng chịu tải lớn, hệ thống có thể triển khai thêm đối tượng được tạo lên dựa trên khuôn mẫu của một lớp của dịch vụ này để phân tán yêu cầu. Tương tự, các dịch vụ ít chịu tải hơn có thể giữ nguyên quy mô nhằm tiết kiệm tài nguyên.

Bên cạnh khả năng mở rộng, hệ thống phân tán cần có khả năng chịu lỗi. Trong thực tế, lỗi có thể xảy ra ở nhiều cấp độ: một dịch vụ có thể bị dừng, một cơ sở dữ liệu có thể phản hồi chậm, một lời gọi HTTP có thể bị quá hạn thời gian chờ hoặc một thông điệp bất đồng bộ có thể xử lý thất bại. Nếu hệ thống không có cơ chế kiểm soát lỗi phù hợp, sự cố ở một thành phần có thể lan truyền sang các thành phần khác và gây ra lỗi dây chuyền. Do đó, các cơ chế như cấu hình thời gian chờ, cơ chế thử lại, cơ chế ngắt mạch và hàng đợi thư chết thường được sử dụng để giúp hệ thống phản ứng tốt hơn trước các tình huống bất thường. [3]

Cơ chế ngắt mạch là một cơ chế thường dùng trong giao tiếp giữa các dịch vụ. Khi một dịch vụ phụ thuộc liên tục trả lỗi hoặc phản hồi quá chậm, circuit breaker có

thể tạm thời ngắt việc gọi tới dịch vụ đó để tránh làm cạn kiệt tài nguyên của dịch vụ gọi. Sau một khoảng thời gian nhất định, hệ thống có thể thử gọi lại để kiểm tra khả năng phục hồi. Cơ chế này giúp hạn chế lỗi dây chuyền và làm cho hệ thống ổn định hơn trong môi trường có nhiều phụ thuộc mạng.

1.3. Kiến trúc microservices và vai trò của API Gateway

Microservices là một phong cách kiến trúc trong đó hệ thống được chia thành nhiều dịch vụ nhỏ, mỗi dịch vụ đảm nhiệm một chức năng nghiệp vụ cụ thể và có thể được phát triển, kiểm thử, triển khai độc lập. Khác với kiến trúc nguyên khối, nơi các module cùng nằm trong một ứng dụng lớn, microservices tổ chức hệ thống thành các thành phần có ranh giới rõ ràng. Mỗi dịch vụ có thể có mã nguồn, cấu hình, cơ sở dữ liệu và vòng đời triển khai riêng.

Trong hệ thống thương mại điện tử, việc tách dịch vụ theo miền nghiệp vụ giúp hệ thống rõ ràng hơn về trách nhiệm. Dịch vụ người dùng tập trung vào thông tin tài khoản, dịch vụ sản phẩm quản lý danh mục hàng hóa, dịch vụ giỏ hàng xử lý trạng thái mua sắm tạm thời, còn dịch vụ đơn hàng xử lý nghiệp vụ đặt hàng. Cách phân chia này phù hợp với nguyên tắc “mỗi dịch vụ chịu trách nhiệm cho một miền nghiệp vụ”, giúp giảm sự phụ thuộc trực tiếp giữa các phần của hệ thống.

Tuy nhiên, khi số lượng dịch vụ tăng lên, client không nên trực tiếp gọi tới từng dịch vụ riêng lẻ. Nếu client phải biết địa chỉ và cách gọi của từng dịch vụ, cấu trúc phía client sẽ trở nên phức tạp, đồng thời hệ thống khó thay đổi về sau. API Gateway được sử dụng để giải quyết vấn đề này. Nó đóng vai trò như một lớp trung gian giữa client và các dịch vụ phía sau. Tất cả yêu cầu từ client đi qua API Gateway, sau đó gateway định tuyến đến dịch vụ phù hợp dựa trên đường dẫn, phương thức HTTP hoặc cấu hình định tuyến.

API Gateway mang lại nhiều lợi ích cho hệ thống microservices. Thứ nhất, nó cung cấp một điểm truy cập thống nhất, giúp client không cần quan tâm đến vị trí cụ thể của từng service. Thứ hai, gateway có thể gom các xử lý chung như định tuyến, kiểm soát CORS, xác thực, phân quyền, ghi log, giám sát hoặc giới hạn tốc độ truy cập. Thứ ba, API Gateway giúp che giấu cấu trúc nội bộ của hệ thống, nhờ đó các dịch vụ phía sau có thể được thay đổi hoặc mở rộng mà không làm thay đổi quá nhiều phía client.

Tuy vậy, API Gateway cũng cần được thiết kế cẩn thận. Nếu toàn bộ lưu lượng đều đi qua gateway, thành phần này có thể trở thành điểm nghẽn hoặc điểm lỗi đơn nếu không được triển khai dự phòng. Trong môi trường thực tế, API Gateway thường cần được nhân bản, cân bằng tải và giám sát chặt chẽ. Ngoài ra, các chính sách bảo mật như xác thực người dùng, kiểm tra token, phân quyền theo vai trò và giới hạn truy cập cũng thường được đặt ở lớp gateway hoặc kết hợp giữa gateway và từng service nghiệp vụ.

1.4. Mô hình database-per-service

Trong hệ thống nguyên khối, các module thường dùng chung một cơ sở dữ liệu. Điều này giúp giao dịch nội bộ dễ xử lý hơn, nhưng lại tạo ra sự phụ thuộc chặt chẽ giữa các module. Khi chuyển sang kiến trúc microservices, việc nhiều service cùng truy cập trực tiếp vào một cơ sở dữ liệu dùng chung có thể làm giảm tính độc lập của từng service. Một thay đổi trong cấu trúc bảng có thể ảnh hưởng đến nhiều service, khiến hệ thống khó bảo trì và khó mở rộng.

Mô hình database-per-service được sử dụng để giải quyết vấn đề này. Theo mô hình này, mỗi service sở hữu cơ sở dữ liệu riêng và chỉ service đó được phép truy cập trực tiếp vào dữ liệu của mình. Các service khác nếu cần thông tin phải giao tiếp thông qua API hoặc thông qua sự kiện. Cách tổ chức này giúp ranh giới sở hữu dữ liệu trở nên rõ ràng hơn, phù hợp với tinh thần của microservices. [4]

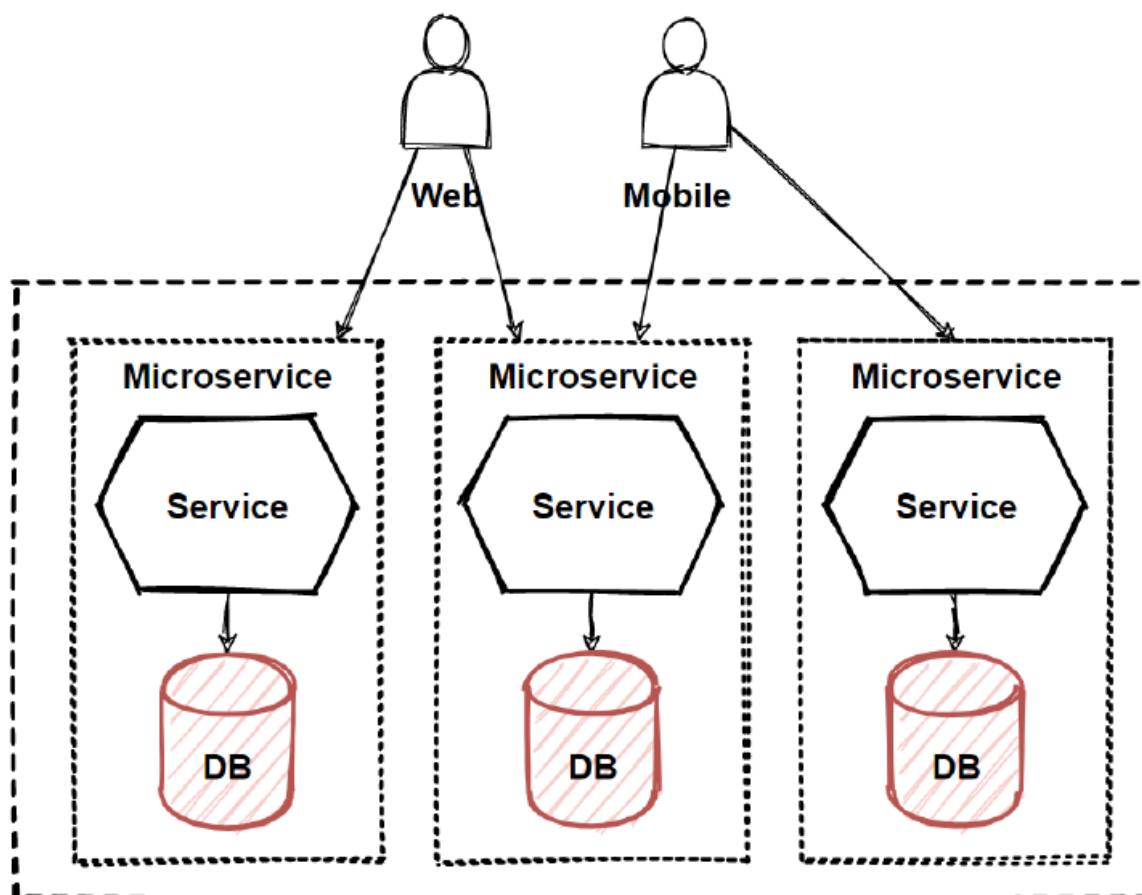
Đối với hệ thống thương mại điện tử, mô hình database-per-service có thể được áp dụng bằng cách tách dữ liệu người dùng, dữ liệu sản phẩm, dữ liệu giỏ hàng và dữ liệu đơn hàng thành các kho lưu trữ riêng. Dịch vụ người dùng quản lý bảng người dùng, dịch vụ sản phẩm quản lý bảng sản phẩm, dịch vụ giỏ hàng quản lý dữ liệu giỏ hàng, còn dịch vụ đơn hàng quản lý dữ liệu đơn hàng và chi tiết đơn hàng. Nhờ vậy, mỗi service có thể phát triển schema riêng phù hợp với nghiệp vụ của mình mà không phụ thuộc trực tiếp vào schema của service khác.

Lợi ích lớn nhất của database-per-service là tăng tính độc lập và khả năng bảo trì. Khi một service cần thay đổi cấu trúc dữ liệu, phạm vi ảnh hưởng được giới hạn trong service đó. Ngoài ra, mỗi service có thể lựa chọn công nghệ lưu trữ phù hợp với nhu cầu riêng nếu hệ thống phát triển ở quy mô lớn hơn. Ví dụ, dữ liệu sản phẩm có thể cần tối ưu cho truy vấn tìm kiếm, dữ liệu đơn hàng cần đảm bảo tính bền vững giao dịch, còn dữ liệu giỏ hàng có thể ưu tiên tốc độ truy cập.

Tuy nhiên, database-per-service cũng làm cho việc đảm bảo tính nhất quán dữ liệu trở nên phức tạp hơn. Do mỗi service chỉ quản lý dữ liệu của mình, một nghiệp vụ trải dài qua nhiều service không thể dễ dàng được xử lý bằng một giao dịch ACID duy nhất như trong hệ thống nguyên khối. Chẳng hạn, khi người dùng đặt hàng, hệ thống có thể cần đồng thời tạo đơn hàng, giảm số lượng tồn kho, cập nhật lịch sử người dùng và xóa giỏ hàng. Nếu tất cả dữ liệu nằm trong một cơ sở dữ liệu, các thao tác này có thể được đặt trong cùng một transaction. Nhưng khi dữ liệu thuộc nhiều service khác nhau, hệ thống cần sử dụng các cơ chế như sự kiện bất đồng bộ, saga hoặc nhất quán cuối cùng để đồng bộ trạng thái.

Vì vậy, database-per-service thường đi kèm với kiến trúc hướng sự kiện. Khi một service hoàn thành một nghiệp vụ quan trọng, nó có thể phát ra sự kiện để các service khác xử lý phần việc liên quan. Ví dụ, sau khi đơn hàng được tạo, hệ thống có thể phát sự kiện “đơn hàng đã được tạo” để dịch vụ sản phẩm cập nhật tồn kho, dịch vụ người dùng cập nhật thông kê đơn hàng và dịch vụ giỏ hàng xóa các sản phẩm đã mua. Cách làm này giúp giảm sự phụ thuộc giữa các service, nhưng đồng thời yêu cầu hệ thống phải xử lý tốt các tình huống như message bị lỗi, message bị xử lý lặp hoặc một service tiêu thụ sự kiện bị tạm dừng.

Như vậy, database-per-service không chỉ là một lựa chọn về lưu trữ dữ liệu, mà còn kéo theo nhiều vấn đề thiết kế liên quan đến giao tiếp, đồng bộ và xử lý lỗi trong hệ thống phân tán. Khi sử dụng mô hình này, cần đánh đổi giữa tính độc lập của từng service và độ phức tạp trong việc đảm bảo dữ liệu nhất quán trên toàn hệ thống. Đây là một trong những nội dung quan trọng nhất khi phân tích các hệ thống microservices trong thực tế.



Hình 1.2: database-per-service trong microservices [5].

1.5. Giao tiếp REST đồng bộ và hạn chế của mô hình gọi liên dịch vụ theo chuỗi

Trong kiến trúc microservices, các dịch vụ thường cần giao tiếp với nhau để hoàn thành một nghiệp vụ tổng thể. Một trong những hình thức giao tiếp phổ biến nhất là REST/HTTP. Đây là cơ chế giao tiếp đồng bộ, trong đó một dịch vụ gửi yêu cầu đến dịch vụ khác và chờ phản hồi trước khi tiếp tục xử lý. REST được sử dụng rộng rãi vì có cấu trúc đơn giản, dễ triển khai, dễ kiểm thử và phù hợp với nhiều hệ thống web hiện đại.

Trong một hệ thống thương mại điện tử, giao tiếp REST đồng bộ thường xuất hiện ở nhiều lớp khác nhau. Ở lớp bên ngoài, trình duyệt hoặc giao diện người dùng gửi

yêu cầu đến API Gateway. API Gateway sau đó định tuyến yêu cầu đến dịch vụ nghiệp vụ phù hợp. Ở lớp bên trong, các dịch vụ cũng có thể gọi lẫn nhau để lấy thông tin cần thiết cho quá trình xử lý. Ví dụ, khi xử lý đơn hàng, dịch vụ đơn hàng có thể cần truy vấn dịch vụ giỏ hàng để lấy danh sách sản phẩm người dùng đã chọn, sau đó truy vấn dịch vụ sản phẩm để kiểm tra thông tin sản phẩm, giá bán và số lượng tồn kho. [6]

Ưu điểm của giao tiếp REST đồng bộ là luồng xử lý rõ ràng. Dịch vụ gọi có thể nhận phản hồi trực tiếp từ dịch vụ được gọi và dựa vào kết quả đó để quyết định bước tiếp theo. Điều này phù hợp với các nghiệp vụ bắt buộc phải có dữ liệu ngay lập tức trước khi trả kết quả cho người dùng. Chẳng hạn, trước khi tạo đơn hàng, hệ thống cần biết giỏ hàng có hợp lệ hay không, sản phẩm còn tồn kho hay không và tổng giá trị đơn hàng là bao nhiêu. Những thông tin này thường cần được kiểm tra trong cùng một luồng xử lý đặt hàng.

Tuy nhiên, giao tiếp REST đồng bộ cũng làm phát sinh một số hạn chế đặc trưng của hệ thống phân tán. Khi một dịch vụ phụ thuộc phản hồi chậm hoặc không khả dụng, dịch vụ gọi cũng bị ảnh hưởng trực tiếp. Nếu không có cơ chế giới hạn thời gian chờ, các request có thể bị treo trong thời gian dài, làm chiếm tài nguyên xử lý và khiến hệ thống phản hồi chậm hơn. Trong trường hợp có nhiều dịch vụ gọi nối tiếp nhau, lỗi ở một dịch vụ có thể lan truyền sang các dịch vụ khác, tạo ra hiện tượng lỗi dây chuyền.

Một vấn đề khác là số lượng lời gọi liên dịch vụ có thể tăng nhanh khi nghiệp vụ trở nên phức tạp. Với luồng đặt hàng, nếu mỗi sản phẩm trong giỏ hàng đều cần một lần gọi riêng đến dịch vụ sản phẩm, số lượng request nội bộ sẽ tăng theo số lượng mặt hàng trong giỏ. Khi người dùng có nhiều sản phẩm trong một đơn hàng, mô hình này có thể làm tăng độ trễ, tăng tải mạng nội bộ và khiến dịch vụ điều phối đơn hàng trở thành điểm nghẽn. Đây là một dạng hạn chế thường gặp trong microservices, đặc biệt khi các lời gọi REST được tổ chức theo chuỗi dài.

Để giảm rủi ro từ giao tiếp đồng bộ, hệ thống cần kết hợp thêm các cơ chế bảo vệ như timeout, retry có kiểm soát, circuit breaker và fallback. Timeout giúp giới hạn thời gian chờ phản hồi từ dịch vụ phụ thuộc. Retry có thể được sử dụng khi lỗi mang tính tạm thời, nhưng cần giới hạn số lần thử lại để tránh làm tăng tải hệ thống. Circuit breaker giúp tạm thời ngắt các lời gọi đến một dịch vụ đang lỗi liên tục, từ đó hạn chế lỗi dây chuyền. Fallback có thể giúp hệ thống trả về thông báo phù hợp hoặc chuyển sang phương án xử lý thay thế khi dịch vụ phụ thuộc chưa sẵn sàng.

Như vậy, REST đồng bộ vẫn là một cơ chế quan trọng trong hệ thống microservices, đặc biệt đối với các nghiệp vụ cần phản hồi tức thời. Tuy nhiên, khi sử dụng REST cho giao tiếp liên dịch vụ, cần thiết kế cẩn thận để tránh phụ thuộc quá chặt giữa các thành phần. Trong các nghiệp vụ không yêu cầu phản hồi ngay lập tức, hệ thống có thể kết hợp thêm truyền thông bất đồng bộ thông qua message broker nhằm giảm coupling và tăng khả năng chịu lỗi.

1.6. RabbitMQ, exchange, queue, routing key và mô hình publish/subscribe

Bên cạnh giao tiếp REST đồng bộ, các hệ thống phân tán hiện đại thường sử dụng cơ chế truyền thông bất đồng bộ để giảm sự phụ thuộc trực tiếp giữa các dịch vụ. Trong mô hình này, một dịch vụ không nhất thiết phải gọi trực tiếp dịch vụ khác và chờ phản hồi ngay lập tức. Thay vào đó, dịch vụ có thể phát sinh một sự kiện hoặc thông điệp, sau đó các dịch vụ quan tâm sẽ nhận và xử lý thông điệp đó theo khả năng của mình. Cách tiếp cận này đặc biệt phù hợp với các tác vụ hậu xử lý, chẳng hạn cập nhật tồn kho, ghi nhận lịch sử giao dịch hoặc gửi thông báo sau khi đơn hàng được tạo.

RabbitMQ là một message broker phổ biến, triển khai giao thức AMQP và cho phép các thành phần trong hệ thống trao đổi thông điệp thông qua hàng đợi. Trong RabbitMQ, bên gửi thông điệp được gọi là publisher, còn bên nhận và xử lý thông điệp được gọi là consumer. Publisher không gửi message trực tiếp đến consumer, mà gửi message đến một exchange. Exchange có nhiệm vụ định tuyến message đến một hoặc nhiều queue dựa trên loại exchange, routing key và binding được cấu hình. Consumer sau đó lắng nghe queue và xử lý message khi message được đưa vào hàng đợi. [7]

Các khái niệm cơ bản trong RabbitMQ gồm exchange, queue, routing key và binding. Exchange là nơi tiếp nhận message từ publisher. Queue là nơi lưu trữ message trước khi consumer xử lý. Routing key là khóa định tuyến đi kèm message, giúp exchange xác định message nên được chuyển đến queue nào. Binding là quan hệ liên kết giữa exchange và queue, thường kèm theo một routing key hoặc một mẫu định tuyến cụ thể. Với topic exchange, việc định tuyến có thể dựa trên pattern matching, nhờ đó hệ thống có thể phân phối message linh hoạt cho nhiều nhóm consumer khác nhau.

Trong hệ thống thương mại điện tử, mô hình publish/subscribe có thể được áp dụng cho các nghiệp vụ phát sinh sau khi đơn hàng được tạo. Khi dịch vụ đơn hàng hoàn tất quá trình lưu đơn hàng, nó có thể phát ra một sự kiện biểu thị rằng đơn hàng mới đã được tạo thành công. Các dịch vụ khác như dịch vụ sản phẩm, dịch vụ người dùng và dịch vụ giỏ hàng có thể đăng ký nhận sự kiện này thông qua các queue riêng. Sau đó, mỗi dịch vụ xử lý phần việc thuộc phạm vi trách nhiệm của mình: dịch vụ sản phẩm cập nhật số lượng tồn kho, dịch vụ người dùng cập nhật thống kê đơn hàng, còn dịch vụ giỏ hàng xóa các mặt hàng đã được đặt.

Điểm quan trọng của mô hình này là cùng một sự kiện có thể được nhiều dịch vụ xử lý độc lập. Đây không phải là mô hình competing consumers theo nghĩa nhiều consumer cùng chia nhau xử lý message từ một queue chung. Ngược lại, nó gần với mô hình publish/subscribe hơn, vì mỗi dịch vụ có queue riêng và đều có thể nhận được bản sao của cùng một sự kiện. Nhờ vậy, các tác vụ hậu xử lý sau khi đặt hàng có thể được tách khỏi luồng request chính, giúp hệ thống linh hoạt hơn, dễ mở rộng hơn và giảm sự phụ thuộc trực tiếp giữa các dịch vụ.

Truyền thông bất đồng bộ cũng giúp cải thiện khả năng chịu lỗi. Nếu một dịch vụ tiêu thụ sự kiện tạm thời gặp sự cố, message có thể vẫn được lưu trong queue và xử lý lại khi dịch vụ hoạt động trở lại. Điều này khác với lời gọi REST đồng bộ, nơi dịch vụ gọi thường phải nhận phản hồi ngay trong thời điểm request. Tuy nhiên, truyền thông bất đồng bộ cũng làm phát sinh các vấn đề mới như xử lý message trùng lặp, đảm bảo thứ tự xử lý, theo dõi trạng thái trung gian và đảm bảo tính nhất quán cuối cùng giữa các dịch vụ.

Vì vậy, RabbitMQ và mô hình publish/subscribe không chỉ đóng vai trò là công cụ truyền message, mà còn là một thành phần quan trọng trong thiết kế hệ thống phân tán. Khi được sử dụng hợp lý, message broker giúp tách rời các dịch vụ, hỗ trợ xử lý bất đồng bộ, giảm tải cho luồng request chính và tăng khả năng chịu lỗi của hệ thống. Tuy nhiên, để đạt hiệu quả cao, hệ thống cần kết hợp thêm các cơ chế như retry, dead-letter queue, idempotency và giám sát hàng đợi.

1.7. Dead-letter queue và cơ chế retry

Trong hệ thống truyền thông bất đồng bộ, không phải message nào cũng được xử lý thành công ngay từ lần đầu tiên. Một consumer có thể gặp lỗi do dữ liệu không hợp lệ, cơ sở dữ liệu tạm thời không phản hồi, dịch vụ phụ thuộc bị lỗi hoặc một exception phát sinh trong quá trình xử lý nghiệp vụ. Nếu không có cơ chế kiểm soát phù hợp, các message lỗi có thể bị mất, bị xử lý lặp vô hạn hoặc làm tắc nghẽn hàng đợi chính.

Retry là cơ chế cho phép hệ thống thử xử lý lại một message sau khi lần xử lý trước đó thất bại. Cơ chế này đặc biệt hữu ích đối với các lỗi tạm thời, chẳng hạn như kết nối cơ sở dữ liệu bị gián đoạn trong thời gian ngắn hoặc tài nguyên mạng chưa sẵn sàng. Thay vì kết luận ngay rằng message không thể xử lý, hệ thống có thể thử lại sau một khoảng thời gian nhất định. Nếu lần thử sau thành công, message được xác nhận và loại khỏi hàng đợi.

Tuy nhiên, retry cần được thiết kế có giới hạn. Nếu một message luôn gây lỗi do dữ liệu sai hoặc do logic xử lý không phù hợp, việc retry vô hạn sẽ làm lãng phí tài nguyên và có thể khiến hàng đợi bị nghẽn. Vì vậy, thông thường hệ thống sẽ giới hạn số lần thử lại. Sau khi vượt quá số lần retry cho phép, message sẽ được chuyển sang một hàng đợi đặc biệt gọi là dead-letter queue.

Dead-letter queue là hàng đợi dùng để lưu các message không thể xử lý thành công sau một số lần thử nhất định hoặc sau khi thỏa mãn một điều kiện lỗi cụ thể. Việc chuyển message sang dead-letter queue giúp hệ thống không làm mất message, đồng thời tránh để message lỗi làm ảnh hưởng đến việc xử lý các message hợp lệ khác trong hàng đợi chính. Người vận hành có thể kiểm tra nội dung dead-letter queue để xác định nguyên nhân lỗi, sửa dữ liệu, điều chỉnh logic xử lý hoặc thực hiện replay message khi cần thiết.

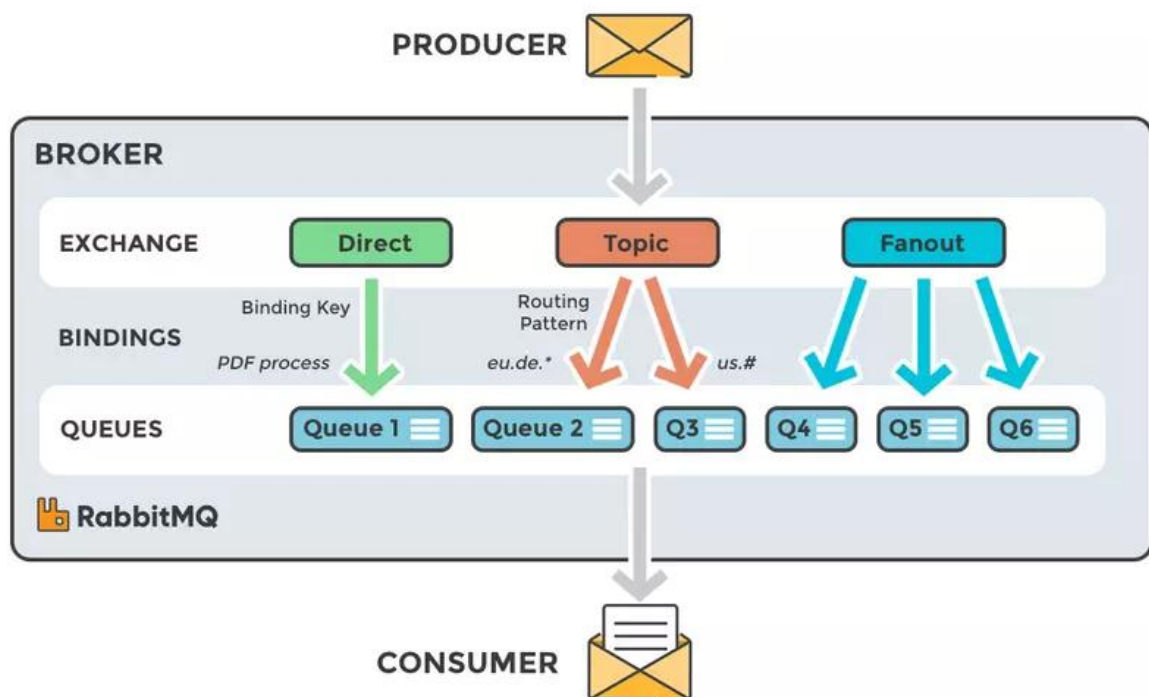
Trong RabbitMQ, dead-letter queue thường được cấu hình thông qua dead-letter exchange và dead-letter routing key. Khi một message bị từ chối, hết hạn hoặc thất bại sau quá trình retry, RabbitMQ có thể chuyển message đó sang exchange dành cho lỗi. Từ exchange này, message được định tuyến tới queue lỗi tương ứng. Cách tổ chức này cho phép từng loại nghiệp vụ có dead-letter queue riêng, giúp việc giám sát và khắc phục sự cố trở nên rõ ràng hơn.

Đối với hệ thống thương mại điện tử phân tán, retry và dead-letter queue có ý nghĩa đặc biệt quan trọng. Ví dụ, nếu sự kiện tạo đơn hàng đã được phát ra nhưng dịch vụ cập nhật tồn kho xử lý thất bại, hệ thống không nên bỏ qua sự kiện này. Nếu message

bị mất, dữ liệu đơn hàng và dữ liệu tồn kho có thể trở nên không nhất quán. Bằng cách sử dụng retry và dead-letter queue, hệ thống có thể giữ lại message lỗi để kiểm tra và xử lý bổ sung, từ đó giảm nguy cơ mất dữ liệu nghiệp vụ quan trọng.

Dù vậy, retry và dead-letter queue không tự động giải quyết toàn bộ bài toán nhất quán dữ liệu. Nếu message được xử lý lại nhiều lần, hệ thống cần đảm bảo thao tác xử lý có tính idempotent, tức là xử lý lặp lại cùng một message không làm sai lệch dữ liệu. Ví dụ, nếu một sự kiện tạo đơn hàng bị xử lý hai lần, hệ thống cần tránh việc trừ tồn kho hai lần hoặc tăng số đơn hàng của người dùng hai lần. Vì vậy, khi thiết kế consumer trong hệ thống event-driven, cần kết hợp retry, dead-letter queue với các cơ chế như event id, bảng ghi nhận message đã xử lý hoặc khóa nghiệp vụ để chống xử lý trùng lặp.

Nhìn chung, retry và dead-letter queue là hai cơ chế quan trọng để tăng độ tin cậy của truyền thông bất đồng bộ. Retry giúp hệ thống tự phục hồi trước các lỗi tạm thời, còn dead-letter queue giúp lưu lại các lỗi không thể xử lý ngay để phục vụ phân tích và khắc phục. Khi được thiết kế hợp lý, hai cơ chế này giúp hệ thống phân tán vận hành ổn định hơn, đặc biệt trong các nghiệp vụ quan trọng như đặt hàng, cập nhật tồn kho và đồng bộ dữ liệu giữa nhiều dịch vụ.



Hình 1.3: Minh họa RabbitMQ exchange, queue, routing key và DLQ [8].

1.8. Kiến trúc hướng sự kiện và tính nhất quán cuối cùng

Trong các hệ thống microservices, mỗi dịch vụ thường đảm nhiệm một miền nghiệp vụ riêng và có thể sở hữu cơ sở dữ liệu độc lập. Cách tổ chức này giúp hệ thống giảm sự phụ thuộc trực tiếp giữa các thành phần, đồng thời tạo điều kiện để từng dịch vụ được phát triển và triển khai độc lập. Tuy nhiên, khi dữ liệu được phân tán ở nhiều

dịch vụ khác nhau, việc thực hiện một giao dịch xuyên suốt toàn bộ hệ thống trở nên phức tạp hơn so với mô hình nguyên khối.

Trong hệ thống nguyên khối, nhiều thao tác nghiệp vụ có thể được đặt trong cùng một transaction ACID của một cơ sở dữ liệu duy nhất. Ví dụ, khi người dùng đặt hàng, hệ thống có thể đồng thời tạo đơn hàng, giảm tồn kho, cập nhật thông tin người dùng và xóa giỏ hàng trong cùng một giao dịch. Nếu một thao tác thất bại, toàn bộ transaction có thể rollback để đảm bảo dữ liệu không bị sai lệch. Tuy nhiên, trong kiến trúc microservices, các dữ liệu này thường thuộc về nhiều dịch vụ và nhiều cơ sở dữ liệu khác nhau. Việc áp dụng một transaction toàn cục trên nhiều dịch vụ là khó triển khai, làm tăng coupling và có thể ảnh hưởng tiêu cực đến khả năng mở rộng của hệ thống.

Kiến trúc hướng sự kiện là một hướng tiếp cận phổ biến để giải quyết bài toán trên. Thay vì buộc tất cả dịch vụ phải cập nhật dữ liệu trong cùng một thời điểm, hệ thống cho phép một dịch vụ phát sinh sự kiện sau khi hoàn thành nghiệp vụ chính. Các dịch vụ khác sẽ lắng nghe sự kiện đó và tự xử lý phần việc thuộc phạm vi trách nhiệm của mình. Ví dụ, sau khi đơn hàng được tạo thành công, hệ thống có thể phát sinh sự kiện “đơn hàng đã được tạo”. Dịch vụ sản phẩm nhận sự kiện để cập nhật tồn kho, dịch vụ người dùng nhận sự kiện để cập nhật thông kê mua hàng, còn dịch vụ giỏ hàng nhận sự kiện để xóa các sản phẩm đã được đặt. [9]

Cách làm này dẫn đến khái niệm tính nhất quán cuối cùng. Tính nhất quán cuối cùng có nghĩa là sau một khoảng thời gian xử lý nhất định, dữ liệu giữa các dịch vụ sẽ đạt đến trạng thái nhất quán, nhưng không nhất thiết phải nhất quán ngay lập tức tại thời điểm request ban đầu hoàn tất. Trong ví dụ đặt hàng, đơn hàng có thể được lưu trước, còn việc cập nhật tồn kho, cập nhật thông tin người dùng và xóa giỏ hàng có thể được xử lý sau thông qua các sự kiện bất đồng bộ. Trong khoảng thời gian ngắn giữa các bước này, hệ thống có thể tồn tại trạng thái trung gian.

Ưu điểm của mô hình nhất quán cuối cùng là giảm sự phụ thuộc giao dịch tức thời giữa các dịch vụ. Dịch vụ tạo đơn hàng không cần giữ kết nối đồng bộ với tất cả các dịch vụ liên quan cho đến khi mọi thao tác hoàn tất. Điều này giúp giảm độ phức tạp của luồng request chính, tăng khả năng mở rộng và giúp hệ thống linh hoạt hơn khi một dịch vụ phụ thuộc tạm thời chậm hoặc chưa sẵn sàng. Đồng thời, message broker có thể lưu trữ sự kiện để các dịch vụ tiêu thụ xử lý sau.

Tuy nhiên, eventual consistency cũng đặt ra nhiều yêu cầu bổ sung. Trước hết, hệ thống cần có cơ chế theo dõi trạng thái trung gian, vì dữ liệu giữa các dịch vụ không luôn luôn đồng bộ tức thời. Thứ hai, hệ thống cần đảm bảo các sự kiện quan trọng không bị mất. Thứ ba, nếu một sự kiện bị xử lý nhiều lần, các thao tác cập nhật dữ liệu phải được thiết kế sao cho không gây sai lệch. Đây là lý do các khái niệm như idempotency, retry, dead-letter queue, event id và cơ chế replay thường đi kèm với kiến trúc hướng sự kiện.

Trong các hệ thống thương mại điện tử, eventual consistency là một lựa chọn thực tế và thường được chấp nhận, miễn là các trạng thái trung gian được kiểm soát tốt. Người dùng cần nhận được phản hồi rõ ràng khi đặt hàng, trong khi các tác vụ hậu xử lý có thể được thực hiện bất đồng bộ. Điều quan trọng là hệ thống phải có cơ chế phát

hiện, ghi nhận và khắc phục khi một tác vụ hậu xử lý thất bại. Nếu không, hệ thống có thể rơi vào trạng thái đơn hàng đã được tạo nhưng tồn kho chưa giảm, giỏ hàng chưa được xóa hoặc thống kê người dùng chưa được cập nhật.

Như vậy, kiến trúc hướng sự kiện và tính nhất quán cuối cùng là hai nội dung có quan hệ chặt chẽ trong hệ thống microservices. Chúng cho phép hệ thống giảm coupling và tăng khả năng mở rộng, nhưng đồng thời yêu cầu người thiết kế phải quan tâm nhiều hơn đến xử lý lỗi, chống xử lý trùng lặp, giám sát hàng đợi và khôi phục dữ liệu khi xảy ra sự cố.

1.9. Tracing và observability trong hệ thống phân tán

Khi hệ thống được chia thành nhiều dịch vụ độc lập, việc theo dõi một request từ đầu đến cuối trở nên khó khăn hơn so với hệ thống nguyên khối. Trong mô hình nguyên khối, một request thường đi qua các module trong cùng một tiến trình, vì vậy việc ghi log và debug tương đối trực tiếp. Ngược lại, trong hệ thống phân tán, một request có thể đi qua API Gateway, nhiều dịch vụ nghiệp vụ, cơ sở dữ liệu và message broker trước khi hoàn tất. Nếu không có cơ chế quan sát phù hợp, việc xác định lỗi xảy ra ở đâu, do thành phần nào và trong bước xử lý nào sẽ rất khó khăn.

Observability là khả năng quan sát và hiểu được trạng thái bên trong của hệ thống thông qua các tín hiệu bên ngoài như log, metrics và traces. Theo OpenTelemetry, ba loại telemetry signal phổ biến gồm traces, metrics và logs; trong đó trace cho biết đường đi của một request khi nó lan truyền qua hệ thống phân tán, metric là phép đo của một service tại thời điểm chạy, còn log là bản ghi dạng văn bản có timestamp để mô tả một sự kiện xảy ra trong hệ thống. Tương tự, Google Cloud cũng xem logging, monitoring và tracing là các năng lực cốt lõi để quan sát, đo lường và chẩn đoán trạng thái hệ thống. Ba nhóm tín hiệu này thường bổ sung cho nhau: log giúp ghi lại các sự kiện và thông báo trong quá trình chạy; metrics giúp theo dõi các chỉ số định lượng như số lượng request, độ trễ, tỷ lệ lỗi, mức sử dụng CPU, RAM hoặc số lượng message trong hàng đợi; traces giúp theo dõi đường đi của một request qua nhiều thành phần khác nhau trong hệ thống. [10], [11]

Trong hệ thống microservices, distributed tracing có vai trò đặc biệt quan trọng. Cơ chế này gán cho mỗi request một định danh truy vết, thường được gọi là trace id. Khi request đi qua nhiều dịch vụ, trace id được truyền tiếp để các log hoặc span ở các dịch vụ khác nhau có thể được liên kết lại. Bên cạnh trace id, span id thường được dùng để biểu diễn từng đoạn xử lý cụ thể trong một request. Nhờ đó, người vận hành có thể biết request đã đi qua những dịch vụ nào, mất bao lâu ở mỗi bước và lỗi phát sinh tại đâu.

Ví dụ, trong một luồng đặt hàng, request có thể bắt đầu từ frontend, đi qua API Gateway, đến dịch vụ đơn hàng, sau đó dịch vụ đơn hàng gọi dịch vụ giỏ hàng và dịch vụ sản phẩm. Sau khi đơn hàng được tạo, hệ thống có thể phát sự kiện qua message broker để các dịch vụ khác tiếp tục xử lý. Nếu không có tracing, khi đơn hàng bị lỗi hoặc dữ liệu không được đồng bộ đầy đủ, người vận hành phải kiểm tra log của nhiều

dịch vụ một cách thủ công. Nếu có trace id thống nhất, quá trình truy vết sẽ thuận lợi hơn nhiều vì các log liên quan có thể được gom lại theo cùng một luồng xử lý.

Observability không chỉ phục vụ việc debug lỗi, mà còn giúp đánh giá chất lượng vận hành của hệ thống. Thông qua metrics, nhóm phát triển có thể theo dõi độ trễ trung bình, độ trễ p95 hoặc p99, tỷ lệ lỗi, số request theo thời gian, số lượng message tồn trong queue, số message đi vào dead-letter queue và số lần circuit breaker mở mạch. Những dữ liệu này rất quan trọng để phát hiện điểm nghẽn, đánh giá khả năng mở rộng và lập kế hoạch tối ưu.

Trong môi trường phát triển hoặc đồ án, hệ thống có thể mới dừng ở mức ghi log kèm trace id và span id. Đây đã là một bước khởi đầu có giá trị, vì nó giúp liên kết các dòng log giữa nhiều dịch vụ. Tuy nhiên, trong môi trường vận hành nghiêm túc, hệ thống nên có thêm backend quan sát tập trung như Zipkin, Jaeger, Grafana Tempo, Prometheus hoặc Grafana. Các công cụ này giúp trực quan hóa trace, theo dõi metrics theo thời gian thực và cảnh báo khi hệ thống vượt ngưỡng an toàn.

Nhìn chung, tracing và observability là thành phần không thể thiếu trong hệ thống phân tán. Khi số lượng dịch vụ tăng lên, khả năng “nhìn thấy” toàn bộ luồng xử lý trở thành điều kiện quan trọng để đảm bảo hệ thống có thể vận hành, bảo trì và khắc phục sự cố hiệu quả. Một kiến trúc microservices không chỉ cần chia dịch vụ hợp lý, mà còn cần được thiết kế để có thể quan sát được trong quá trình hoạt động.

1.10. Docker Compose trong triển khai hệ thống nhiều dịch vụ

Docker Compose là công cụ cho phép định nghĩa và chạy nhiều container như một ứng dụng thống nhất. Thay vì phải khởi động từng container bằng nhiều lệnh riêng lẻ, người phát triển có thể mô tả toàn bộ hệ thống trong một file cấu hình, thường là docker-compose.yml. File này quy định các service cần chạy, image hoặc Dockerfile tương ứng, cổng ánh xạ, biến môi trường, volume, network và quan hệ phụ thuộc giữa các container.

Trong bối cảnh hệ thống microservices, Docker Compose đặc biệt hữu ích cho môi trường phát triển, thử nghiệm và trình diễn đồ án. Một hệ thống phân tán thường bao gồm nhiều thành phần như frontend, API Gateway, các dịch vụ nghiệp vụ, message broker và nhiều cơ sở dữ liệu khác nhau. Nếu khởi động thủ công từng thành phần, quá trình thiết lập môi trường sẽ tốn thời gian và dễ sai sót. Docker Compose giúp chuẩn hóa quá trình này, cho phép khởi động toàn bộ topology bằng một lệnh duy nhất.

Khi dùng Docker Compose, mỗi service có thể được đóng gói trong một container riêng. Các container trong cùng một compose project thường được kết nối với nhau qua một network nội bộ. Nhờ đó, các service có thể gọi nhau bằng tên service thay vì phải dùng địa chỉ IP cố định. Ví dụ, một dịch vụ đơn hàng có thể gọi dịch vụ sản phẩm thông qua hostname nội bộ tương ứng, còn các dịch vụ backend có thể kết nối đến cơ sở dữ liệu của mình bằng tên container hoặc tên service được định nghĩa trong compose.

Docker Compose cũng hỗ trợ khai báo quan hệ phụ thuộc giữa các service thông qua thuộc tính `depends_on`. Thuộc tính này giúp kiểm soát thứ tự khởi động ở mức container. Chẳng hạn, service backend có thể được cấu hình phụ thuộc vào database hoặc message broker. Tuy nhiên, cần phân biệt giữa việc “container đã được khởi động” và việc “dịch vụ bên trong container đã sẵn sàng xử lý request”. Một database container có thể đã chạy nhưng chưa sẵn sàng nhận kết nối, hoặc một message broker có thể cần thêm thời gian để khởi tạo. Vì vậy, trong các hệ thống nghiêm túc hơn, cần kết hợp `depends_on` với healthcheck hoặc cơ chế retry kết nối trong ứng dụng.

Đối với báo cáo môn học, Docker Compose có giá trị ở chỗ nó giúp mô phỏng một hệ thống phân tán ngay trên một máy phát triển. Mặc dù các container có thể cùng chạy trên một máy tính, chúng vẫn là các tiến trình độc lập, giao tiếp với nhau qua mạng container. Điều này giúp người học quan sát được nhiều đặc điểm thực tế của hệ thống phân tán như giao tiếp qua HTTP, kết nối cơ sở dữ liệu riêng, message broker, log từng service và lỗi khởi động theo phụ thuộc.

Tuy nhiên, Docker Compose không nên được hiểu là giải pháp thay thế hoàn chỉnh cho các nền tảng orchestration trong môi trường production. Compose phù hợp với môi trường local, demo, kiểm thử tích hợp hoặc triển khai nhỏ. Khi hệ thống cần tự động mở rộng, tự phục hồi, rolling update, service discovery động, quản lý secret, cân bằng tải nâng cao hoặc triển khai trên nhiều node vật lý, các nền tảng như Kubernetes thường phù hợp hơn. Do đó, khi đánh giá một hệ thống dùng Docker Compose, cần nhìn nhận đây là mô hình triển khai cục bộ phục vụ phát triển và thử nghiệm, không phải hạ tầng production hoàn chỉnh.

Như vậy, Docker Compose là một công cụ hữu ích để đóng gói và tái lập môi trường chạy cho hệ thống microservices. Nó giúp giảm độ phức tạp khi khởi động nhiều thành phần, hỗ trợ mô phỏng topology phân tán và tạo điều kiện thuận lợi cho việc kiểm thử. Tuy nhiên, để hệ thống tiến gần hơn đến môi trường triển khai thực tế, cần bổ sung thêm healthcheck, volume bền vững, quản lý cấu hình an toàn, giám sát tập trung và cơ chế orchestration mạnh hơn.

Kết luận chương

Chương 1 đã trình bày các cơ sở lý thuyết quan trọng liên quan đến việc xây dựng hệ thống thương mại điện tử phân tán theo kiến trúc microservices. Các nội dung đã được trình bày bao gồm khái niệm hệ thống phân tán, tính trong suốt, khả năng mở rộng, khả năng chịu lỗi, kiến trúc microservices, API Gateway, mô hình database-per-service, giao tiếp REST đồng bộ, message broker, RabbitMQ, retry, dead-letter queue, kiến trúc hướng sự kiện, eventual consistency, tracing, observability và Docker Compose.

Những nội dung lý thuyết này tạo nền tảng để phân tích hệ thống ở các chương tiếp theo. Cụ thể, kiến trúc microservices và API Gateway giúp làm rõ cách tổ chức các dịch vụ nghiệp vụ và điểm vào thống nhất cho client. Mô hình database-per-service giúp giải thích cách dữ liệu được tách theo từng miền nghiệp vụ. REST và RabbitMQ cho thấy sự kết hợp giữa giao tiếp đồng bộ và bất đồng bộ. Retry, dead-letter queue và circuit

breaker là cơ sở để đánh giá khả năng chịu lỗi. Eventual consistency giúp giải thích cách hệ thống xử lý dữ liệu phân tán khi không sử dụng transaction toàn cục. Cuối cùng, tracing, observability và Docker Compose là các yếu tố quan trọng trong triển khai, theo dõi và kiểm thử hệ thống nhiều dịch vụ.

Từ cơ sở đó, chương tiếp theo sẽ đi vào phân tích bài toán và thiết kế hệ thống cụ thể, bao gồm kiến trúc tổng thể, các thành phần chức năng, luồng dữ liệu, API, cơ sở dữ liệu, cơ chế truyền thông giữa các dịch vụ và các điểm mạnh, điểm yếu của giải pháp.

CHƯƠNG 2: PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ HỆ THỐNG

2.1. Nhận diện bài toán và phạm vi hệ thống

Về mặt thiết kế hệ thống phân tán, bài toán đặt ra bao gồm nhiều yêu cầu quan trọng. Thứ nhất, hệ thống cần có một điểm truy cập thống nhất để phía người dùng không phải giao tiếp trực tiếp với từng dịch vụ riêng lẻ. Thứ hai, mỗi dịch vụ cần có phạm vi dữ liệu riêng để đảm bảo tính độc lập và giảm phụ thuộc giữa các miền nghiệp vụ. Thứ ba, các thao tác hậu xử lý sau khi tạo đơn hàng, chẳng hạn như cập nhật tồn

kho, cập nhật thông tin người dùng và xóa giỏ hàng, cần được tách khỏi luồng request chính để giảm coupling giữa các dịch vụ. Thứ tư, hệ thống cần có cơ chế xử lý lỗi ở mức nhất định, đặc biệt trong các tình huống dịch vụ phụ thuộc phản hồi chậm, message xử lý thất bại hoặc một thành phần tạm thời không khả dụng.

Từ các yêu cầu trên và cơ sở lý thuyết đã được xác định cụ thể ở chương 1, có thể xác định bản chất của hệ thống là một nền tảng thương mại điện tử phân tán theo kiến trúc microservices, kết hợp giữa giao tiếp REST đồng bộ và truyền thông bất đồng bộ thông qua message broker. Trọng tâm kỹ thuật của đề tài là tổ chức hệ thống nhiều dịch vụ, tách dữ liệu theo từng service, sử dụng API Gateway làm điểm vào thống nhất, dùng RabbitMQ để xử lý sự kiện sau khi tạo đơn hàng và đánh giá các vấn đề đặc trưng của hệ thống phân tán như tính nhất quán cuối cùng, khả năng chịu lỗi, khả năng mở rộng và bảo mật.

Việc xác định đúng phạm vi bài toán là rất quan trọng để tránh đánh giá sai bản chất hệ thống. Nếu nhìn dưới góc độ ứng dụng web, hệ thống cung cấp các chức năng thương mại điện tử cơ bản. Nếu nhìn dưới góc độ môn Các hệ thống phân tán, hệ thống là một ví dụ về cách nhiều dịch vụ độc lập phối hợp với nhau để tạo thành một ứng dụng thống nhất đối với người dùng cuối. Đây là cơ sở để các phần tiếp theo phân tích sâu hơn về kiến trúc tổng thể, các thành phần chức năng, luồng dữ liệu, giao thức truyền thông và các cơ chế đảm bảo độ tin cậy của hệ thống.

2.2. Kiến trúc tổng thể của hệ thống

Hệ thống được thiết kế theo kiến trúc nhiều lớp, trong đó phía người dùng tương tác với giao diện web, giao diện web gửi yêu cầu đến API Gateway, sau đó API Gateway định tuyến yêu cầu đến các dịch vụ nghiệp vụ tương ứng. Các dịch vụ nghiệp vụ chính bao gồm dịch vụ người dùng, dịch vụ sản phẩm, dịch vụ giỏ hàng và dịch vụ đơn hàng. Mỗi dịch vụ chịu trách nhiệm cho một miền nghiệp vụ riêng và có cơ sở dữ liệu độc lập. Ngoài ra, hệ thống sử dụng RabbitMQ làm message broker để truyền sự kiện giữa các dịch vụ sau khi đơn hàng được tạo.

Ở lớp giao diện, frontend được triển khai như một ứng dụng web tĩnh, phục vụ các thao tác của người dùng và quản trị viên. Người dùng có thể đăng ký, đăng nhập, xem danh sách sản phẩm, thêm sản phẩm vào giỏ hàng, đặt hàng và xem lịch sử đơn hàng. Quản trị viên có thể thao tác với các chức năng quản lý như thêm, sửa, xóa sản phẩm và xem dữ liệu tổng quan của hệ thống. Frontend không giao tiếp trực tiếp với từng dịch vụ nghiệp vụ riêng lẻ mà gửi các request đến API Gateway.

API Gateway đóng vai trò là điểm vào thống nhất của hệ thống. Thành phần này tiếp nhận request từ frontend và định tuyến đến service phù hợp dựa trên đường dẫn API. Chẳng hạn, các request liên quan đến người dùng được chuyển đến dịch vụ người dùng, request liên quan đến sản phẩm được chuyển đến dịch vụ sản phẩm, request liên quan đến giỏ hàng được chuyển đến dịch vụ giỏ hàng và request liên quan đến đơn hàng

được chuyển đến dịch vụ đơn hàng. Cách tổ chức này giúp che giấu cấu trúc bên trong của hệ thống, làm cho phía client chỉ cần biết một điểm truy cập duy nhất.

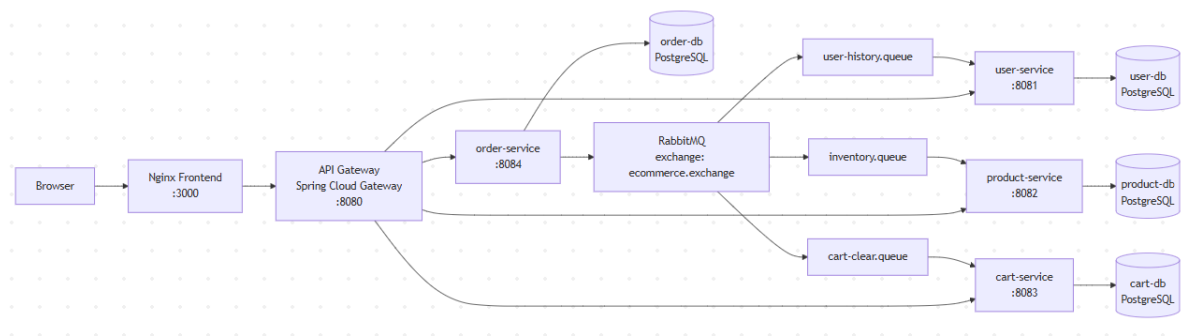
Ở lớp nghiệp vụ, mỗi service đảm nhiệm một vai trò cụ thể. Dịch vụ người dùng xử lý các chức năng đăng ký, đăng nhập và quản lý thông tin người dùng. Dịch vụ sản phẩm quản lý danh mục sản phẩm, thông tin giá bán, mô tả và số lượng tồn kho. Dịch vụ giỏ hàng lưu trữ các sản phẩm mà người dùng đã chọn trước khi đặt hàng. Dịch vụ đơn hàng đóng vai trò điều phối luồng checkout, bao gồm lấy thông tin giỏ hàng, kiểm tra thông tin sản phẩm, tính tổng giá trị đơn hàng, lưu đơn hàng và phát sinh sự kiện sau khi đơn hàng được tạo.

Ở lớp dữ liệu, hệ thống áp dụng mô hình database-per-service. Mỗi dịch vụ nghiệp vụ có cơ sở dữ liệu riêng, không truy cập trực tiếp vào cơ sở dữ liệu của dịch vụ khác. Cách thiết kế này giúp đảm bảo ranh giới sở hữu dữ liệu giữa các service, giảm phụ thuộc trực tiếp ở tầng lưu trữ và phù hợp với tinh thần của kiến trúc microservices. Tuy nhiên, việc tách dữ liệu cũng làm phát sinh yêu cầu đồng bộ giữa các dịch vụ, đặc biệt trong các nghiệp vụ liên quan đến đặt hàng.

RabbitMQ được sử dụng để xử lý các tác vụ bất đồng bộ sau khi đơn hàng được tạo. Khi dịch vụ đơn hàng hoàn tất việc lưu đơn hàng, hệ thống phát ra sự kiện tạo đơn hàng. Các dịch vụ khác nhận sự kiện này thông qua các queue riêng và thực hiện phần việc tương ứng. Dịch vụ sản phẩm cập nhật số lượng tồn kho, dịch vụ người dùng cập nhật thống kê đơn hàng, còn dịch vụ giỏ hàng xóa các mặt hàng đã được đặt. Cách làm này giúp tách các tác vụ hậu xử lý khỏi luồng request chính, đồng thời thể hiện rõ mô hình event-driven và eventual consistency trong hệ thống.

Về mặt triển khai cục bộ, hệ thống có thể được tổ chức thành nhiều tiến trình hoặc container khác nhau. Frontend, API Gateway, các service nghiệp vụ, RabbitMQ và các cơ sở dữ liệu PostgreSQL là những thành phần độc lập, giao tiếp với nhau qua mạng nội bộ. Cách tổ chức này giúp mô phỏng một môi trường phân tán ở quy mô nhỏ, phù hợp cho việc nghiên cứu, kiểm thử và đánh giá các đặc điểm của hệ thống microservices.

Tổng thể, kiến trúc của hệ thống thể hiện sự kết hợp giữa hai kiểu giao tiếp. Kiểu thứ nhất là giao tiếp đồng bộ qua REST/HTTP, được sử dụng trong các request cần phản hồi trực tiếp như đăng nhập, lấy danh sách sản phẩm, thêm sản phẩm vào giỏ hàng hoặc tạo đơn hàng. Kiểu thứ hai là giao tiếp bất đồng bộ qua RabbitMQ, được sử dụng cho các tác vụ hậu xử lý sau khi đơn hàng đã được tạo. Sự kết hợp này giúp hệ thống vừa đảm bảo được các nghiệp vụ cần phản hồi tức thời, vừa giảm sự phụ thuộc trực tiếp giữa các dịch vụ trong các thao tác không nhất thiết phải hoàn tất ngay lập tức.



Hình 2.1: Sơ đồ tổng quan của hệ thống thương mại điện tử theo kiến trúc Microservices

Sơ đồ trên mô tả kiến trúc tổng thể của hệ thống thương mại điện tử phân tán theo mô hình microservices. Ở lớp ngoài cùng, người dùng tương tác với hệ thống thông qua trình duyệt và giao diện web. Giao diện này không gọi trực tiếp đến từng dịch vụ nghiệp vụ riêng lẻ, mà gửi toàn bộ yêu cầu đến API Gateway. API Gateway đóng vai trò là điểm vào thống nhất, tiếp nhận request từ phía client và định tuyến đến dịch vụ phù hợp ở phía sau.

Bên dưới API Gateway là các dịch vụ nghiệp vụ độc lập, bao gồm dịch vụ người dùng, dịch vụ sản phẩm, dịch vụ giỏ hàng và dịch vụ đơn hàng. Mỗi dịch vụ đảm nhiệm một miền chức năng riêng, có cổng xử lý riêng và được triển khai như một tiến trình độc lập. Cách tổ chức này giúp hệ thống giảm sự phụ thuộc giữa các thành phần, đồng thời tạo điều kiện để từng dịch vụ có thể được phát triển, kiểm thử, triển khai và mở rộng một cách tương đối độc lập.

Ở tầng lưu trữ, hệ thống áp dụng mô hình database-per-service. Mỗi dịch vụ nghiệp vụ gắn với một cơ sở dữ liệu PostgreSQL riêng, chẳng hạn dịch vụ người dùng quản lý dữ liệu người dùng, dịch vụ sản phẩm quản lý dữ liệu sản phẩm, dịch vụ giỏ hàng quản lý dữ liệu giỏ hàng và dịch vụ đơn hàng quản lý dữ liệu đơn hàng. Việc tách riêng cơ sở dữ liệu giúp ranh giới sở hữu dữ liệu rõ ràng hơn, hạn chế việc các dịch vụ truy cập trực tiếp vào dữ liệu của nhau và phù hợp với nguyên tắc thiết kế của kiến trúc microservices.

Ngoài giao tiếp REST đồng bộ giữa client, gateway và các dịch vụ nghiệp vụ, hệ thống còn sử dụng RabbitMQ để xử lý các tác vụ bất đồng bộ sau khi đơn hàng được tạo. Khi dịch vụ đơn hàng hoàn tất quá trình tạo đơn, sự kiện liên quan đến đơn hàng được gửi đến message broker. Từ đó, các hàng đợi tương ứng sẽ chuyển sự kiện đến các dịch vụ cần xử lý tiếp theo, chẳng hạn cập nhật tồn kho ở dịch vụ sản phẩm, cập nhật thông tin mua hàng ở dịch vụ người dùng và xóa giỏ hàng ở dịch vụ giỏ hàng.

Nhìn tổng thể, sơ đồ thể hiện rõ hai đặc điểm quan trọng của hệ thống phân tán. Thứ nhất, hệ thống che giấu sự phức tạp bên trong thông qua API Gateway, giúp phía client chỉ cần làm việc với một điểm truy cập duy nhất. Thứ hai, hệ thống kết hợp giao tiếp đồng bộ và bất đồng bộ để vừa đảm bảo phản hồi trực tiếp cho các thao tác chính, vừa giảm sự phụ thuộc giữa các dịch vụ trong các tác vụ hậu xử lý. Đây là cách tổ chức phù hợp với các hệ thống microservices, đặc biệt trong những bài toán có nhiều miền nghiệp vụ cần phối hợp như thương mại điện tử.



Hình 2.2. Các công nghệ chính được sử dụng trong hệ thống

Bảng 2.1. Các thành phần và cơ sở dữ liệu của hệ thống

Thành phần	Cổng	Công nghệ sử dụng	CSDL liên quan	Vai trò
Frontend	3000	Nginx, HTML, CSS, Vanilla JS	Không có DB riêng	Giao diện người dùng và trang admin
API Gateway	8080	Spring Boot, Spring Cloud Gateway	Không có DB riêng	Điểm vào thống nhất, route request
user-service	8081	Spring Boot, Spring Web, Spring Data JPA, AMQP	user-db	Đăng ký, đăng nhập, truy vấn user, tăng totalOrders
product-service	8082	Spring Boot, Spring Web, Spring Data JPA, AMQP	product-db	CRUD sản phẩm, tìm kiếm theo tên, giảm tồn kho
cart-service	8083	Spring Boot, Spring Web, Spring Data JPA, AMQP	cart-db	Thêm giỏ hàng, lấy giỏ theo user, xóa giỏ sau checkout

order-service	8084	Spring Boot, Spring Web, JPA, AMQP, Resilience4j	order-db	Tạo đơn hàng, phối hợp gọi service khác, publish event
RabbitMQ	5672	RabbitMQ 3 Management	ecommerce.exchange, các queue và DLQ	Message broker cho order.created
user-db	5432	PostgreSQL 15	Bảng users	Lưu người dùng
product-db	5433	PostgreSQL 15	Bảng products	Lưu sản phẩm
cart-db	5434	PostgreSQL 15	Bảng cart_items	Lưu giỏ hàng
order-db	5435	PostgreSQL 15	Bảng orders, order_items	Lưu đơn hàng

Bảng 2.1 trình bày các thành phần chính cấu thành nên hệ thống thương mại điện tử phân tán. Các thành phần được phân loại theo vai trò trong kiến trúc tổng thể, bao gồm lớp giao diện người dùng, lớp định tuyến trung gian, các dịch vụ nghiệp vụ, message broker và các cơ sở dữ liệu tương ứng. Việc mô tả các thành phần theo công triển khai, công nghệ sử dụng, kho dữ liệu hoặc hàng đợi liên quan giúp làm rõ cách hệ thống được tổ chức và vận hành trong môi trường nhiều dịch vụ.

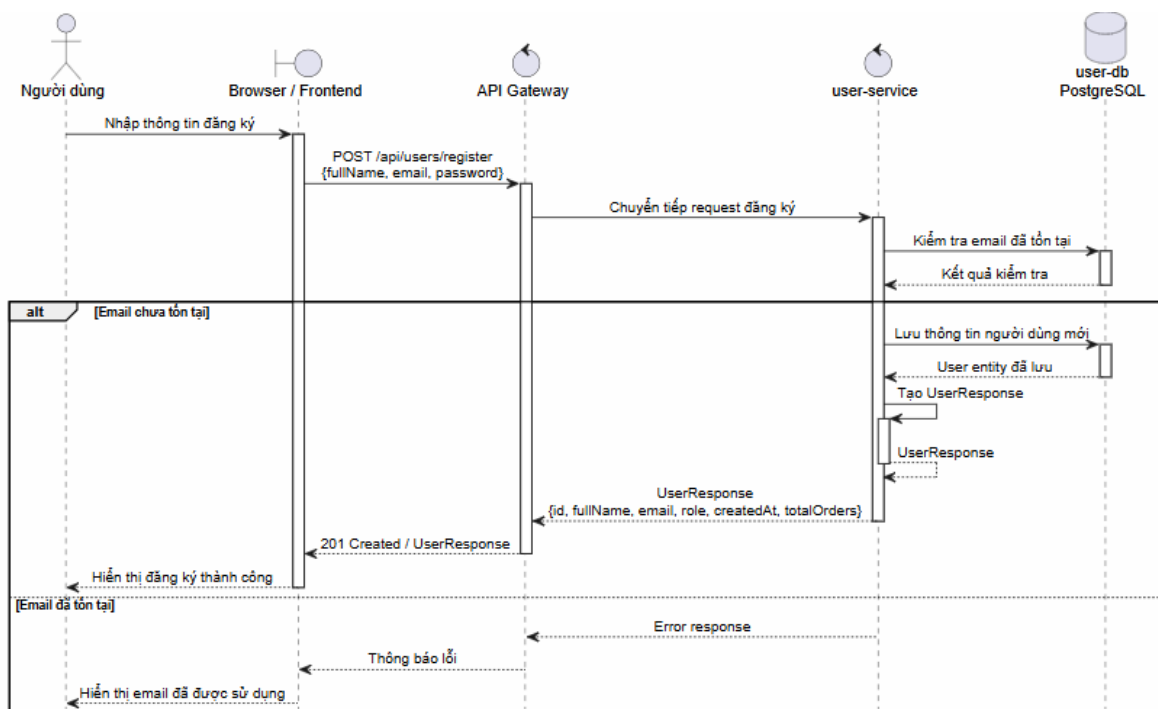
Qua bảng này có thể thấy hệ thống được chia thành nhiều service độc lập theo từng miền nghiệp vụ. Frontend đảm nhiệm vai trò giao diện tương tác với người dùng và quản trị viên. API Gateway đóng vai trò là điểm vào thống nhất, tiếp nhận request từ phía client và chuyển tiếp đến service phù hợp. Các service nghiệp vụ như user-service, product-service, cart-service và order-service lần lượt xử lý các chức năng liên quan đến người dùng, sản phẩm, giỏ hàng và đơn hàng. Bên cạnh đó, RabbitMQ được sử dụng làm message broker để hỗ trợ truyền thông bất đồng bộ sau khi đơn hàng được tạo.

Một điểm quan trọng trong thiết kế là mỗi service nghiệp vụ có cơ sở dữ liệu riêng. Cách tổ chức này phù hợp với mô hình database-per-service trong kiến trúc microservices, giúp tăng tính độc lập giữa các service và hạn chế việc truy cập trực tiếp vào dữ liệu của nhau. Đồng thời, việc tách riêng các cơ sở dữ liệu cũng làm rõ trách nhiệm sở hữu dữ liệu của từng thành phần trong hệ thống. Cho thấy hệ thống không được tổ chức như một ứng dụng nguyên khối, mà được chia thành nhiều thành phần độc lập có nhiệm vụ rõ ràng. Các service giao tiếp với nhau thông qua API Gateway, REST API và message broker thay vì truy cập trực tiếp vào dữ liệu nội bộ của nhau. Đây là cơ sở quan trọng để phân tích các đặc trưng phân tán của hệ thống như tách miền nghiệp vụ, tách sở hữu dữ liệu, truyền thông đồng bộ – bất đồng bộ, khả năng mở rộng và khả năng chịu lỗi.

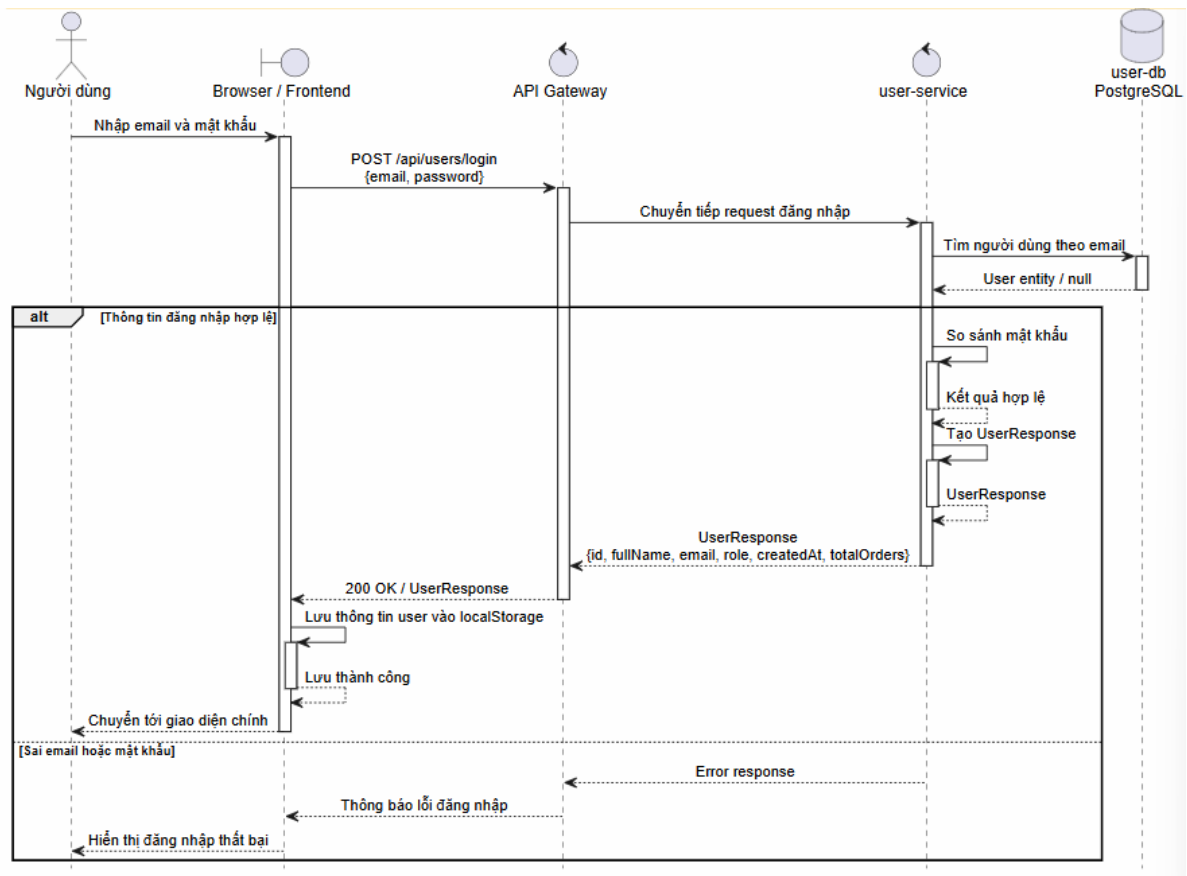
2.3. Các sơ đồ tuần tự của hệ thống

Khi xem xét hệ thống ở góc độ luồng dữ liệu tổng thể, có thể nhận thấy các chức năng chính được tổ chức thành nhiều chuỗi xử lý liên kết với nhau. Mỗi chuỗi xử lý bắt đầu từ thao tác của người dùng trên giao diện web, đi qua API Gateway, sau đó được chuyển tiếp đến service nghiệp vụ phù hợp. Tùy theo loại nghiệp vụ, service có thể truy cập cơ sở dữ liệu riêng, gọi sang service khác thông qua REST API hoặc phát sinh sự kiện bất đồng bộ thông qua RabbitMQ.

Luồng thứ nhất và thứ hai là đăng ký và đăng nhập người dùng. Người dùng nhập thông tin tài khoản trên giao diện web, sau đó frontend gửi request đến API Gateway. Gateway định tuyến request đến user-service, nơi chịu trách nhiệm xử lý thông tin người dùng. Service này kiểm tra dữ liệu, truy cập cơ sở dữ liệu user-db, lưu hoặc truy vấn thông tin tài khoản và trả về dữ liệu phản hồi cho frontend. Sau khi đăng nhập thành công, frontend lưu thông tin người dùng ở phía client để phục vụ các thao tác tiếp theo như thêm sản phẩm vào giỏ hàng, đặt hàng hoặc xem lịch sử mua hàng.

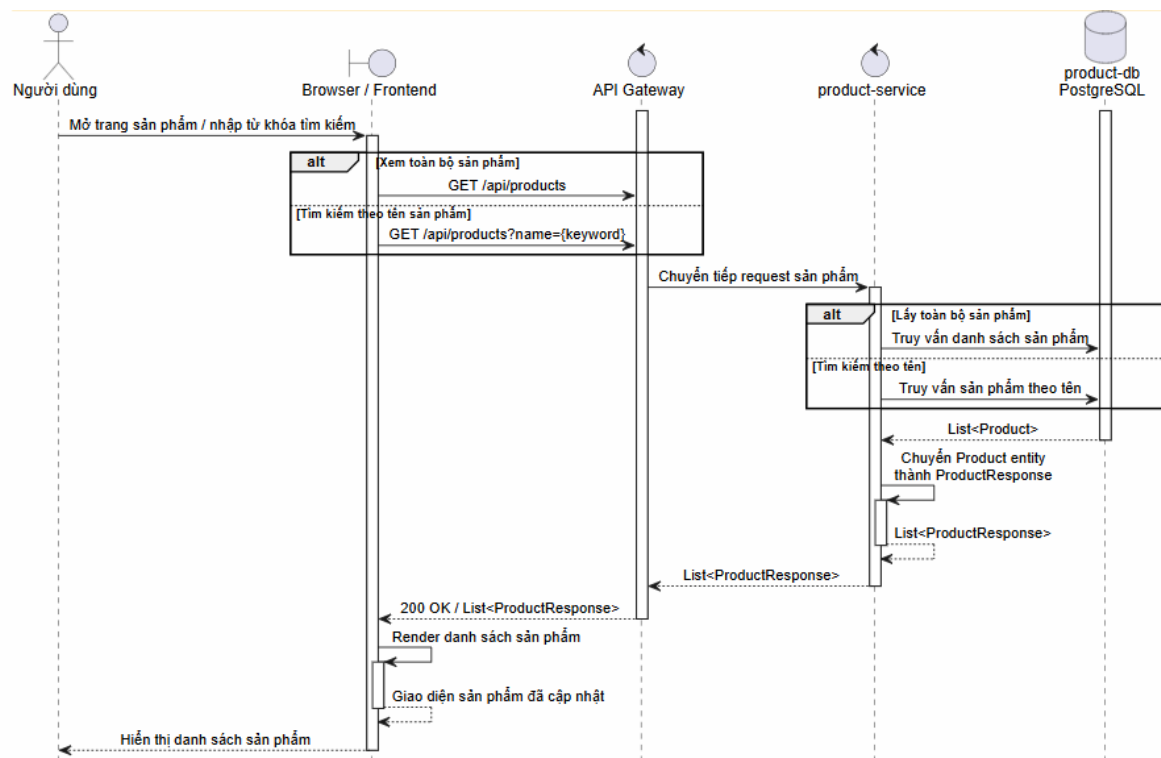


Hình 2.3: Sơ đồ tuần tự cho luồng đăng ký người dùng



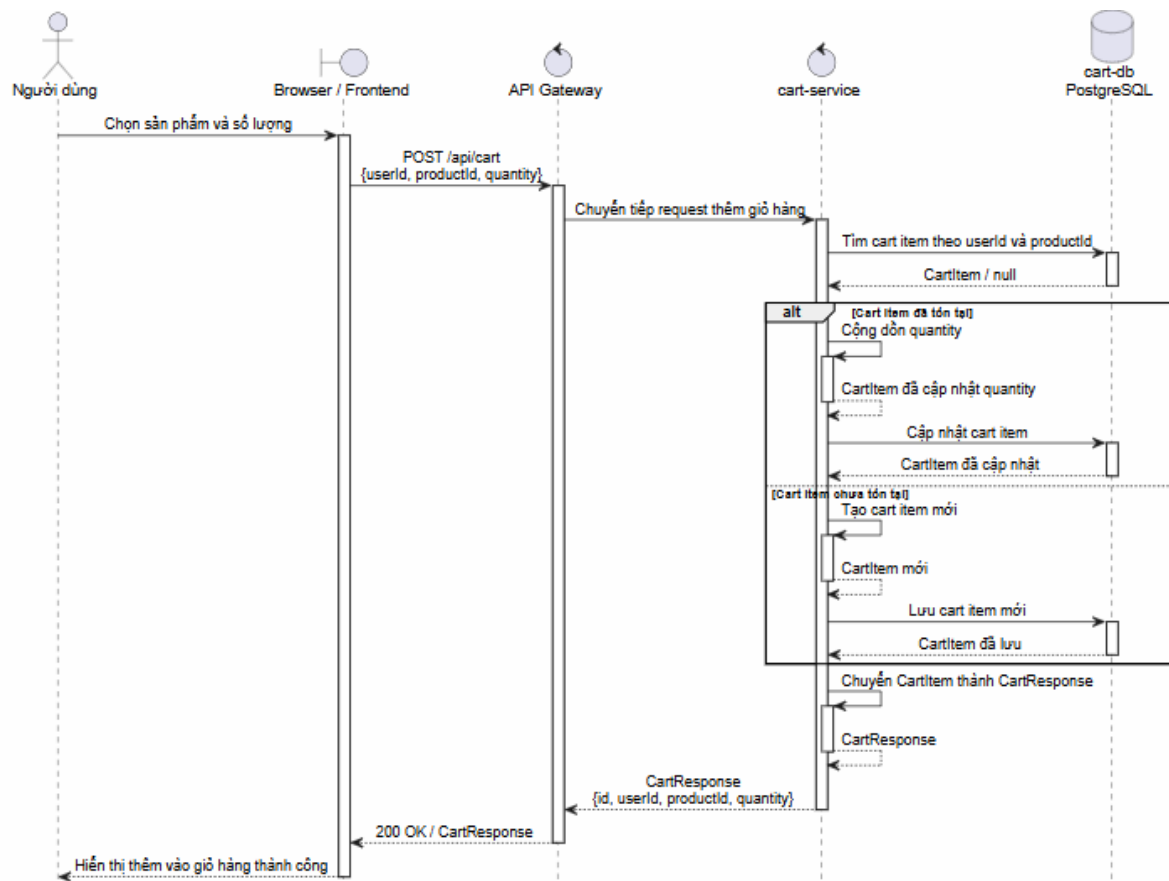
Hình 2.4: Sơ đồ tuần tự cho luồng đăng nhập người dùng

Luồng thứ ba là duyệt và tìm kiếm sản phẩm. Khi người dùng truy cập trang sản phẩm hoặc nhập từ khóa tìm kiếm, frontend gửi request đến API Gateway theo nhóm endpoint sản phẩm. Gateway chuyển tiếp request đến product-service. Service này truy vấn cơ sở dữ liệu product-db để lấy danh sách sản phẩm, chi tiết sản phẩm hoặc kết quả tìm kiếm theo tên. Dữ liệu sản phẩm sau đó được trả về cho frontend để hiển thị trên giao diện người dùng. Đây là một luồng xử lý tương đối trực tiếp, chủ yếu sử dụng giao tiếp REST đồng bộ giữa client, gateway và service sản phẩm.



Hình 2.5: Sơ đồ tuần tự cho luồng duyệt và tìm kiếm sản phẩm

Luồng thứ tư là quản lý giỏ hàng. Khi người dùng thêm một sản phẩm vào giỏ, frontend gửi request chứa các thông tin cần thiết như `userId`, `productId` và `quantity` đến API Gateway. Gateway định tuyến request đến `cart-service`. Service này lưu dữ liệu vào `cart-db`, trong đó mỗi bản ghi thể hiện một sản phẩm thuộc giỏ hàng của một người dùng cụ thể. Nếu sản phẩm đã tồn tại trong giỏ hàng của người dùng, hệ thống không nhất thiết tạo bản ghi mới mà có thể cập nhật số lượng sản phẩm hiện có. Cách xử lý này giúp dữ liệu giỏ hàng gọn hơn và phản ánh đúng hành vi mua sắm của người dùng.



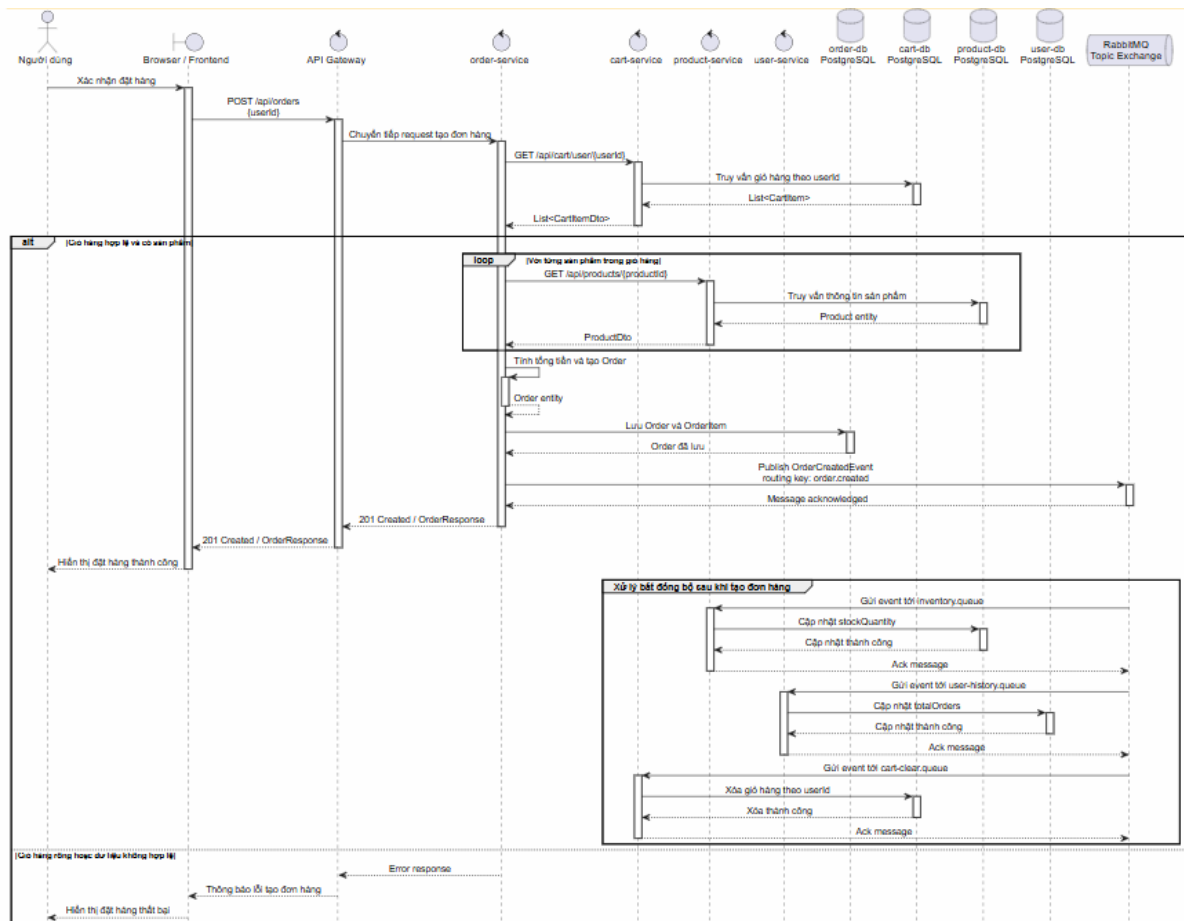
Hình 2.6: Sơ đồ tuần tự cho luồng thêm sản phẩm vào giỏ hàng

Luồng thứ năm và cũng là luồng quan trọng nhất là checkout, tức quá trình tạo đơn hàng. Đây là luồng thể hiện rõ nhất đặc điểm phân tán của hệ thống vì nó kết hợp nhiều service, nhiều cơ sở dữ liệu và cả hai hình thức giao tiếp: đồng bộ qua REST và bất đồng bộ qua RabbitMQ. Khi người dùng xác nhận đặt hàng, frontend gửi request tạo đơn đến API Gateway. Gateway chuyển request đến order-service. Tại đây, order-service đóng vai trò điều phối nghiệp vụ: trước hết gọi cart-service để lấy danh sách sản phẩm trong giỏ hàng của người dùng, sau đó gọi product-service để lấy thông tin chi tiết từng sản phẩm, kiểm tra giá bán và tồn kho.

Sau khi thu thập đủ dữ liệu cần thiết, order-service tính tổng giá trị đơn hàng, tạo bản ghi đơn hàng và chi tiết đơn hàng trong order-db. Đây là phần xử lý đồng bộ vì hệ thống cần phản hồi trực tiếp cho người dùng rằng đơn hàng đã được tạo hay chưa. Tuy nhiên, các thao tác hậu xử lý như giảm tồn kho, cập nhật tổng số đơn hàng của người dùng và xóa giỏ hàng không được thực hiện trực tiếp trong cùng một transaction toàn cục. Thay vào đó, order-service phát sinh sự kiện OrderCreatedEvent đến RabbitMQ để các service liên quan tự xử lý phần việc của mình.

Sau khi sự kiện được đưa vào RabbitMQ, các service tiêu thụ sự kiện thông qua các queue riêng. product-service nhận sự kiện để cập nhật số lượng tồn kho trong product-db. user-service nhận sự kiện để cập nhật thông tin thống kê đơn hàng của người dùng trong user-db. cart-service nhận sự kiện để xóa các sản phẩm trong giỏ hàng của người dùng sau khi đặt hàng thành công. Cách tổ chức này giúp tách các tác vụ hậu xử

lý khởi luồng request chính, giảm sự phụ thuộc trực tiếp giữa các service và phù hợp với mô hình nhất quán cuối cùng trong kiến trúc microservices



Hình 2.7: Sơ đồ tuần tự cho luồng checkout và tạo đơn hàng

Sơ đồ tuần tự trên mô tả luồng checkout và tạo đơn hàng trong hệ thống. Quy trình bắt đầu khi người dùng xác nhận đặt hàng trên giao diện web. Frontend gửi request POST /api/orders kèm userId đến API Gateway, sau đó Gateway chuyển tiếp yêu cầu này đến order-service. Trong luồng xử lý chính, order-service đóng vai trò điều phối nghiệp vụ: trước hết service gọi cart-service để lấy danh sách sản phẩm trong giỏ hàng của người dùng, sau đó với từng sản phẩm trong giỏ hàng, order-service tiếp tục gọi product-service để lấy thông tin sản phẩm cần thiết. Sau khi thu thập đủ dữ liệu, order-service tính tổng giá trị đơn hàng, tạo đối tượng đơn hàng và lưu thông tin Order cùng OrderItem vào order-db.

Sơ đồ cũng thể hiện nhánh xử lý điều kiện của quá trình checkout. Nếu giỏ hàng hợp lệ và có sản phẩm, hệ thống tiếp tục tạo đơn hàng, lưu dữ liệu vào cơ sở dữ liệu và phát sinh sự kiện OrderCreatedEvent tới RabbitMQ với routing key order.created. Sau đó, order-service trả về OrderResponse thông qua API Gateway để frontend hiển thị thông báo đặt hàng thành công cho người dùng. Ngược lại, nếu giỏ hàng rỗng hoặc dữ liệu không hợp lệ, order-service trả về phản hồi lỗi, Gateway chuyển tiếp thông báo này về frontend và người dùng nhận được thông báo đặt hàng thất bại.

Điểm quan trọng của sơ đồ là luồng checkout được chia thành hai phần rõ ràng. Phần thứ nhất là luồng xử lý đồng bộ, bao gồm các bước từ khi người dùng gửi yêu cầu đặt hàng cho đến khi đơn hàng được lưu và hệ thống trả phản hồi cho frontend. Phần này cần hoàn tất trong thời gian request còn đang mở, vì người dùng cần biết thao tác đặt hàng có thành công hay không. Các lời gọi từ order-service đến cart-service và product-service đều thuộc nhóm giao tiếp REST đồng bộ, do đó độ trễ hoặc lỗi ở các service này có thể ảnh hưởng trực tiếp đến quá trình tạo đơn hàng.

Phần thứ hai là luồng xử lý bất đồng bộ sau khi đơn hàng được tạo thành công. Thay vì để order-service tự cập nhật toàn bộ dữ liệu thuộc các miền nghiệp vụ khác, hệ thống phát sự kiện qua RabbitMQ. Từ sự kiện này, product-service nhận message từ inventory.queue để cập nhật số lượng tồn kho trong product-db; user-service nhận message từ user-history.queue để cập nhật tổng số đơn hàng của người dùng trong user-db; và cart-service nhận message từ cart-clear.queue để xóa giỏ hàng của người dùng trong cart-db. Mỗi service xử lý phần việc thuộc phạm vi trách nhiệm của mình và gửi xác nhận xử lý message sau khi hoàn tất.

Về mặt thiết kế hệ thống phân tán, sơ đồ cho thấy hệ thống kết hợp giữa giao tiếp đồng bộ và bất đồng bộ trong cùng một nghiệp vụ. Luồng đồng bộ giúp đảm bảo người dùng nhận được phản hồi rõ ràng về kết quả đặt hàng, còn luồng bất đồng bộ giúp tách các tác vụ hậu xử lý khỏi request chính. Cách tổ chức này làm giảm sự phụ thuộc trực tiếp giữa order-service và các service còn lại, đồng thời phù hợp với mô hình database-per-service và eventual consistency. Trong một khoảng thời gian ngắn sau khi đơn hàng được tạo, dữ liệu tồn kho, thống kê người dùng và giỏ hàng có thể chưa được cập nhật đồng thời ngay lập tức. Tuy nhiên, sau khi các consumer xử lý xong sự kiện, hệ thống sẽ tiến tới trạng thái nhất quán cuối cùng.

Như vậy, luồng dữ liệu đầu cuối của hệ thống không chỉ thể hiện các chức năng nghiệp vụ của một nền tảng thương mại điện tử, mà còn phản ánh rõ các nguyên lý quan trọng của kiến trúc microservices: tách service theo miền nghiệp vụ, tách dữ liệu theo service, sử dụng API Gateway làm điểm vào thống nhất, kết hợp REST đồng bộ với message broker bất đồng bộ và chấp nhận eventual consistency trong các nghiệp vụ trải dài qua nhiều thành phần.

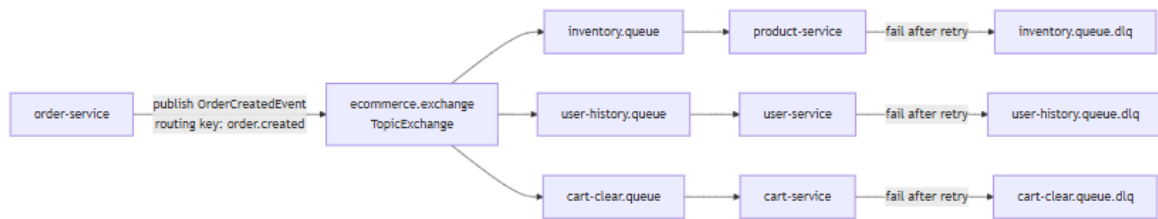
2.4. Thiết kế giao tiếp bất đồng bộ qua RabbitMQ

Sau khi đơn hàng được tạo thành công, hệ thống không xử lý toàn bộ các tác vụ liên quan trong cùng một luồng request đồng bộ. Thay vào đó, hệ thống sử dụng RabbitMQ làm message broker để phát tán sự kiện tạo đơn hàng đến các service liên quan. Cách thiết kế này giúp tách luồng tạo đơn hàng khỏi các tác vụ hậu xử lý như cập nhật tồn kho, cập nhật thống kê người dùng và xóa giỏ hàng sau khi đặt hàng.

Trong kiến trúc này, order-service đóng vai trò là publisher. Khi quá trình tạo đơn hàng hoàn tất, service này phát sinh sự kiện OrderCreatedEvent và gửi sự kiện vào exchange chính có tên ecommerce.exchange với routing key order.created. RabbitMQ

sau đó định tuyến message đến các queue đã được binding với routing key tương ứng. Mỗi queue đại diện cho một nhóm xử lý nghiệp vụ riêng, giúp các service có thể nhận cùng một sự kiện nhưng xử lý theo trách nhiệm độc lập của mình.

Cụ thể, sự kiện tạo đơn hàng được phân phối đến ba queue chính. Queue `inventory.queue` được sử dụng để chuyển sự kiện đến `product-service`, phục vụ việc cập nhật số lượng tồn kho. Queue `user-history.queue` chuyển sự kiện đến `user-service`, phục vụ việc cập nhật thông tin thống kê đơn hàng của người dùng. Queue `cart-clear.queue` chuyển sự kiện đến `cart-service`, phục vụ việc xóa giỏ hàng sau khi đơn hàng đã được tạo thành công. Nhờ đó, cùng một sự kiện nghiệp vụ có thể được nhiều service xử lý mà không cần `order-service` gọi trực tiếp từng service trong luồng request chính.



Hình 2.8: Sơ đồ luồng sự kiện RabbitMQ sau khi tạo đơn hàng

Sơ đồ trên cho thấy RabbitMQ hoạt động như một lớp trung gian giữa service phát sinh sự kiện và các service tiêu thụ sự kiện. `order-service` không gửi trực tiếp message đến từng service đích, mà chỉ publish message vào exchange. Exchange chịu trách nhiệm định tuyến message đến các queue dựa trên routing key và binding đã được cấu hình. Mô hình này giúp giảm coupling giữa các service, vì publisher không cần biết chi tiết các consumer đang tồn tại, số lượng consumer là bao nhiêu hoặc mỗi consumer xử lý sự kiện theo cách nào.

Về mặt mô hình truyền thông, cách tổ chức này gần với publish/subscribe hơn là mô hình competing consumers. Trong competing consumers, nhiều consumer thường cùng đọc từ một queue và mỗi message chỉ được một consumer xử lý. Ngược lại, ở luồng xử lý sau khi tạo đơn hàng, mỗi service có queue riêng và đều nhận được bản sao của sự kiện `OrderCreatedEvent`. Điều này là cần thiết vì các service không thay thế nhau, mà mỗi service xử lý một phần nghiệp vụ khác nhau: sản phẩm cập nhật tồn kho, người dùng cập nhật thống kê đơn hàng, còn giỏ hàng được xóa sau khi đặt hàng.

Một điểm quan trọng khác trong thiết kế là cơ chế xử lý lỗi bằng retry và dead-letter queue. Trong thực tế, consumer có thể xử lý message thất bại do nhiều nguyên nhân như lỗi kết nối cơ sở dữ liệu, dữ liệu không hợp lệ, service chưa sẵn sàng hoặc lỗi logic phát sinh trong quá trình xử lý. Nếu message lỗi bị bỏ qua, dữ liệu giữa các service có thể trở nên không nhất quán. Ví dụ, đơn hàng đã được tạo nhưng tồn kho chưa được cập nhật, hoặc giỏ hàng chưa được xóa sau khi đặt hàng. Vì vậy, hệ thống cần cơ chế lưu lại các message xử lý thất bại để có thể kiểm tra và khắc phục sau.

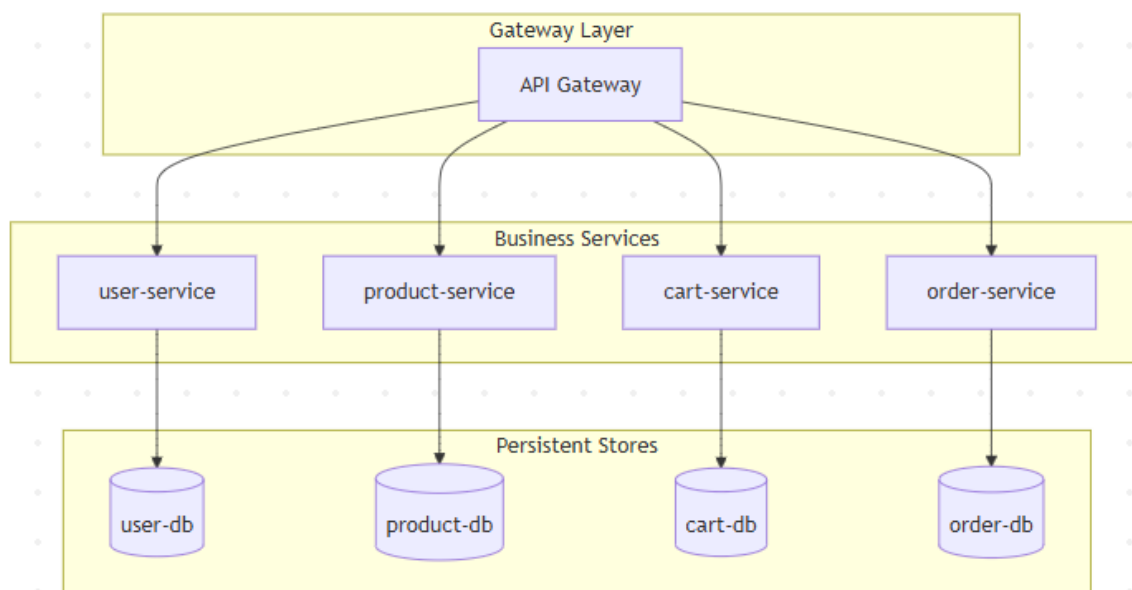
Với mỗi queue xử lý chính, hệ thống thiết kế thêm một dead-letter queue tương ứng. Khi consumer xử lý message thất bại sau số lần thử lại nhất định, message sẽ được chuyển sang queue lỗi thay vì bị mất hoàn toàn. Chẳng hạn, nếu product-service không cập nhật được tồn kho sau nhiều lần thử, message có thể được đưa vào inventory.queue.dlq. Tương tự, message lỗi của user-service được đưa vào user-history.queue.dlq, còn message lỗi của cart-service được đưa vào cart-clear.queue.dlq.

Cơ chế retry giúp hệ thống có khả năng tự phục hồi trước các lỗi tạm thời. Nếu lỗi chỉ xảy ra trong thời gian ngắn, chẳng hạn cơ sở dữ liệu phản hồi chậm hoặc kết nối mạng nội bộ bị gián đoạn, việc thử lại có thể giúp message được xử lý thành công mà không cần can thiệp thủ công. Tuy nhiên, retry cần được giới hạn để tránh tình trạng một message lỗi bị xử lý lặp vô hạn, gây tiêu tốn tài nguyên và làm nghẽn hàng đợi chính. Sau khi vượt quá giới hạn retry, việc chuyển message sang DLQ là lựa chọn phù hợp để cô lập lỗi và bảo vệ luồng xử lý chính.

2.5. Thiết kế dữ liệu theo mô hình database-per-service

Bên cạnh việc tách hệ thống thành nhiều service độc lập, một điểm quan trọng trong thiết kế microservices là cách tổ chức dữ liệu. Trong hệ thống này, mỗi service nghiệp vụ được gắn với một cơ sở dữ liệu riêng, thay vì toàn bộ hệ thống cùng truy cập vào một cơ sở dữ liệu dùng chung. Cách tổ chức này phù hợp với mô hình database-per-service, trong đó mỗi service chịu trách nhiệm sở hữu, quản lý và thay đổi dữ liệu thuộc phạm vi nghiệp vụ của mình.

Cụ thể, user-service quản lý dữ liệu người dùng trong user-db, product-service quản lý dữ liệu sản phẩm trong product-db, cart-service quản lý dữ liệu giỏ hàng trong cart-db, còn order-service quản lý dữ liệu đơn hàng trong order-db. Mỗi service chỉ truy cập trực tiếp vào cơ sở dữ liệu của chính nó. Các service khác nếu cần dữ liệu không truy vấn trực tiếp vào bảng của service đó, mà phải thông qua API hoặc thông qua sự kiện bất đồng bộ. Cách thiết kế này giúp ranh giới trách nhiệm giữa các service rõ ràng hơn và hạn chế sự phụ thuộc trực tiếp ở tầng lưu trữ.



Hình 2.9. Sơ đồ database-per-service của hệ thống

Sơ đồ trên thể hiện quan hệ một-một giữa các service nghiệp vụ và cơ sở dữ liệu tương ứng. API Gateway chỉ đóng vai trò định tuyến request đến các service phía sau, không truy cập trực tiếp vào tầng dữ liệu. Các service nghiệp vụ là thành phần duy nhất làm việc trực tiếp với cơ sở dữ liệu của mình. Điều này giúp hệ thống duy trì nguyên tắc đóng gói dữ liệu: dữ liệu người dùng thuộc về user-service, dữ liệu sản phẩm thuộc về product-service, dữ liệu giỏ hàng thuộc về cart-service, còn dữ liệu đơn hàng thuộc về order-service.

Ưu điểm đầu tiên của mô hình này là tăng tính độc lập giữa các service. Khi một service cần thay đổi cấu trúc bảng, bổ sung trường dữ liệu hoặc tối ưu cách lưu trữ, phạm vi ảnh hưởng chủ yếu nằm trong service đó. Ví dụ, nếu service sản phẩm cần bổ sung thêm thuộc tính liên quan đến mô tả sản phẩm hoặc số lượng tồn kho, thay đổi này không trực tiếp làm ảnh hưởng đến bảng người dùng, bảng giỏ hàng hay bảng đơn hàng. Tương tự, nếu service đơn hàng cần mở rộng thêm trạng thái đơn hàng, chi tiết đơn hàng hoặc thông tin thanh toán trong tương lai, thay đổi đó có thể được thực hiện trong phạm vi order-db.

Ưu điểm thứ hai là mô hình này phù hợp với nguyên tắc phân chia miền nghiệp vụ. Mỗi service không chỉ có mã nguồn và API riêng, mà còn có quyền sở hữu dữ liệu riêng. Nhờ vậy, hệ thống tránh được tình trạng nhiều service cùng truy cập và sửa đổi một bảng dữ liệu chung, vốn dễ dẫn đến phụ thuộc chặt chẽ và khó kiểm soát lỗi. Khi các service không truy cập trực tiếp vào cơ sở dữ liệu của nhau, việc thay đổi nội bộ của một service ít có khả năng làm hỏng service khác.

Tuy nhiên, database-per-service cũng tạo ra những thách thức nhất định. Trong hệ thống nguyên khối, nhiều thao tác nghiệp vụ có thể được thực hiện trong cùng một

transaction ACID của một cơ sở dữ liệu duy nhất. Ngược lại, khi dữ liệu được chia thành nhiều cơ sở dữ liệu riêng, các nghiệp vụ trải dài qua nhiều service sẽ khó đảm bảo tính nhất quán tức thời. Luồng checkout là ví dụ điển hình: hệ thống cần tạo đơn hàng, cập nhật tồn kho, cập nhật thông kê người dùng và xóa giỏ hàng. Các dữ liệu này nằm ở nhiều service khác nhau, vì vậy không thể xử lý đơn giản bằng một transaction cơ sở dữ liệu duy nhất.

Để giải quyết vấn đề trên, hệ thống kết hợp database-per-service với truyền thông bất đồng bộ qua RabbitMQ. order-service lưu đơn hàng vào order-db trước, sau đó phát sinh sự kiện tạo đơn hàng để các service khác xử lý phần dữ liệu của mình. product-service cập nhật tồn kho trong product-db, user-service cập nhật thông tin người dùng trong user-db, còn cart-service xóa giỏ hàng trong cart-db. Cách làm này thể hiện mô hình đồng bộ sự kiện, tức dữ liệu giữa các service có thể chưa nhất quán ngay lập tức tại thời điểm request kết thúc, nhưng sẽ tiến tới trạng thái nhất quán sau khi các consumer xử lý xong sự kiện.

Kết luận chương

Chương 2 đã trình bày quá trình phân tích bài toán và thiết kế hệ thống thương mại điện tử theo kiến trúc microservices. Hệ thống được tổ chức thành nhiều thành phần độc lập, gồm giao diện người dùng, API Gateway, các service nghiệp vụ và các cơ sở dữ liệu riêng cho từng service. Cách thiết kế này giúp phân tách rõ các miền chức năng như người dùng, sản phẩm, giỏ hàng và đơn hàng, đồng thời tạo điều kiện thuận lợi cho việc bảo trì, mở rộng và kiểm thử từng thành phần.

Chương này cũng đã làm rõ các luồng dữ liệu chính của hệ thống, đặc biệt là luồng checkout. Trong luồng này, order-service đóng vai trò điều phối, gọi đồng bộ đến cart-service và product-service để lấy dữ liệu cần thiết, sau đó lưu đơn hàng vào order-db và phát sinh sự kiện tạo đơn hàng qua RabbitMQ. Các service liên quan tiếp tục xử lý bất đồng bộ để cập nhật tồn kho, cập nhật thông tin người dùng và xóa giỏ hàng.

Bên cạnh đó, chương đã phân tích mô hình database-per-service và cơ chế giao tiếp bất đồng bộ qua RabbitMQ. Mỗi service sở hữu cơ sở dữ liệu riêng, hạn chế truy cập trực tiếp vào dữ liệu của service khác. Điều này phù hợp với kiến trúc microservices, nhưng cũng đặt ra yêu cầu xử lý tính nhất quán dữ liệu giữa các service thông qua event-driven architecture và eventual consistency.

Nhìn chung, Chương 2 đã chỉ ra rằng hệ thống không chỉ là một website thương mại điện tử thông thường, mà còn là một mô hình hệ thống phân tán ở quy mô nhỏ, có sự kết hợp giữa API Gateway, REST API, RabbitMQ, database-per-service và xử lý sự kiện. Những phân tích này là cơ sở để Chương 3 tiếp tục trình bày quá trình triển khai, kiểm thử và các hướng cải tiến của hệ thống.

CHƯƠNG 3: TRIỂN KHAI VÀ KẾT QUẢ

Sau khi đã trình bày các cơ sở lý thuyết về hệ thống phân tán, kiến trúc microservices, API Gateway, REST API, RabbitMQ, database-per-service và eventual consistency ở Chương 1, chương 2 đã đi vào phân tích bài toán và thiết kế hệ thống thương mại điện tử theo hướng phân tán. Trên cơ sở đó, chương này tập trung trình bày kết quả triển khai hệ thống, minh họa bằng các hình ảnh thực tế sau khi chạy dự án, đồng thời đánh giá những kết quả đạt được, những hạn chế còn tồn tại và các hướng cải tiến trong tương lai.

Khác với Chương 2, vốn tập trung vào kiến trúc, thành phần, luồng dữ liệu và cơ chế giao tiếp giữa các service, Chương 3 đi theo hướng trực quan hơn. Các nội dung được trình bày xoay quanh những hình ảnh minh họa chính: cây thư mục dự án, giao diện đăng nhập hoặc đăng ký, giao diện danh sách sản phẩm, giao diện giỏ hàng, kết quả tạo đơn hàng thành công, kết quả xử lý sự kiện sau khi tạo đơn và giao diện quản trị sản phẩm. Thông qua các hình ảnh này, người đọc có thể thấy rõ hệ thống đã được triển khai thành một ứng dụng có thể sử dụng được, đồng thời vẫn phản ánh đúng các nguyên lý của hệ thống phân tán đã phân tích ở các chương trước.

Về mặt nội dung, Chương 3 không chỉ nhằm chứng minh rằng hệ thống có giao diện và chức năng cơ bản, mà còn làm rõ cách các chức năng đó được tổ chức theo mô hình nhiều service. Một thao tác đơn giản ở phía người dùng, chẳng hạn đăng nhập, xem sản phẩm, thêm sản phẩm vào giỏ hoặc đặt hàng, thực chất là kết quả phối hợp giữa frontend, API Gateway, các service nghiệp vụ, cơ sở dữ liệu riêng và RabbitMQ. Đây là điểm khác biệt quan trọng giữa một website thương mại điện tử thông thường và một hệ thống thương mại điện tử được thiết kế theo kiến trúc microservices.

3.1. Cấu trúc triển khai của dự án

Hệ thống được triển khai theo kiến trúc microservices, trong đó mỗi thành phần đảm nhiệm một vai trò riêng biệt. Thay vì đặt toàn bộ mã nguồn trong một ứng dụng duy nhất, dự án được chia thành nhiều module độc lập như api-gateway, user-service, product-service, cart-service, order-service và frontend. Mỗi module đại diện cho một phần chức năng hoặc một lớp trong kiến trúc tổng thể của hệ thống.

API Gateway là thành phần trung gian giữa frontend và các service nghiệp vụ. Thành phần này chịu trách nhiệm tiếp nhận request từ phía người dùng, sau đó định tuyến request đến service phù hợp. Nhờ có API Gateway, frontend không cần biết địa chỉ cụ thể của từng service phía sau. Điều này giúp che giấu cấu trúc nội bộ của hệ thống, đồng thời làm cho phía client chỉ cần giao tiếp với một điểm vào thống nhất.

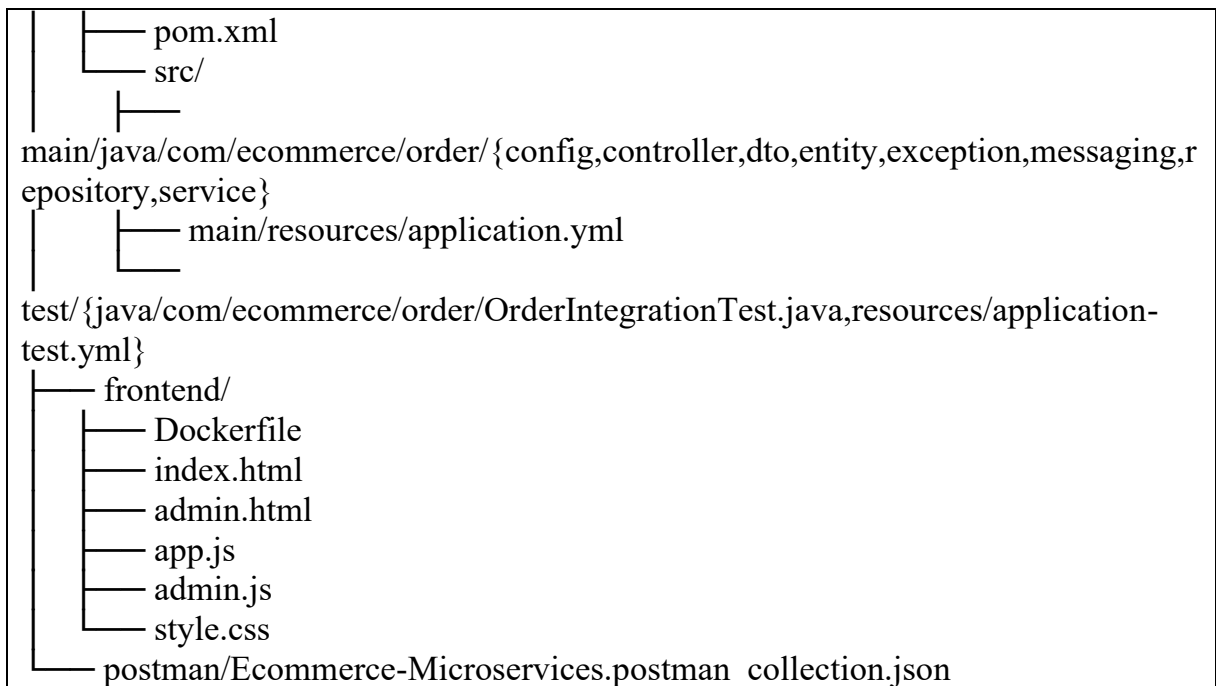
Các service nghiệp vụ được tách theo miền chức năng. user-service phụ trách các nghiệp vụ liên quan đến người dùng như đăng ký, đăng nhập và truy vấn thông tin người dùng. product-service phụ trách quản lý thông tin sản phẩm, bao gồm danh sách sản phẩm, chi tiết sản phẩm và thao tác quản trị sản phẩm. cart-service phụ trách lưu trữ và quản lý giỏ hàng của người dùng. order-service phụ trách xử lý nghiệp vụ đặt hàng, tính tổng tiền, lưu đơn hàng và phát sinh sự kiện sau khi đơn hàng được tạo.

Frontend là thành phần giao diện, phục vụ cả người dùng thông thường và quản trị viên. Người dùng có thể thao tác với sản phẩm, giỏ hàng và đơn hàng thông qua giao diện web. Quản trị viên có thể sử dụng giao diện admin để quản lý sản phẩm. Việc tách frontend thành một phần riêng giúp hệ thống phân chia rõ ràng giữa lớp trình bày và lớp xử lý nghiệp vụ.

```

cachethongphantan/
├── README.md
├── docker-compose.yml
├── api-gateway/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/resources/application.yml
├── user-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/
│       └──
│           ├── java/com/ecommerce/user/{config,controller,dto,entity,exception,messaging,repository,service}
│           └── resources/application.yml
├── product-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/
│       └──
│           ├── java/com/ecommerce/product/{config,controller,dto,entity,exception,messaging,repository,service}
│           └── resources/application.yml
├── cart-service/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/main/
│       └──
│           ├── java/com/ecommerce/cart/{controller,dto,entity,exception,messaging,repository,service}
│           └── resources/application.yml
├── order-service/
│   └── Dockerfile

```



Hình 3.1: Cây thư mục của dự án

Hình 3.1 minh họa cây thư mục của dự án khi được mở trong terminal. Qua hình này, có thể quan sát được cách dự án được chia thành nhiều module thay vì tổ chức như một ứng dụng nguyên khối. Đây là một minh chứng trực quan cho việc hệ thống được xây dựng theo hướng microservices. Mỗi service có mã nguồn, cấu hình và logic xử lý riêng, giúp việc phát triển và bảo trì từng phần trở nên thuận lợi hơn.

Việc tổ chức thư mục theo từng service cũng có ý nghĩa quan trọng trong quá trình triển khai và kiểm thử. Khi một service gặp lỗi, nhóm phát triển có thể tập trung kiểm tra trong phạm vi service đó thay vì phải tìm kiếm trong toàn bộ mã nguồn lớn. Khi cần bổ sung chức năng mới, việc xác định module cần thay đổi cũng rõ ràng hơn. Ví dụ, nếu cần thay đổi cách tính tổng tiền đơn hàng, nhóm sẽ tập trung vào order-service; nếu cần thay đổi giao diện hiển thị sản phẩm, nhóm sẽ chỉnh sửa frontend; nếu cần thay đổi dữ liệu sản phẩm, nhóm sẽ làm việc với product-service.

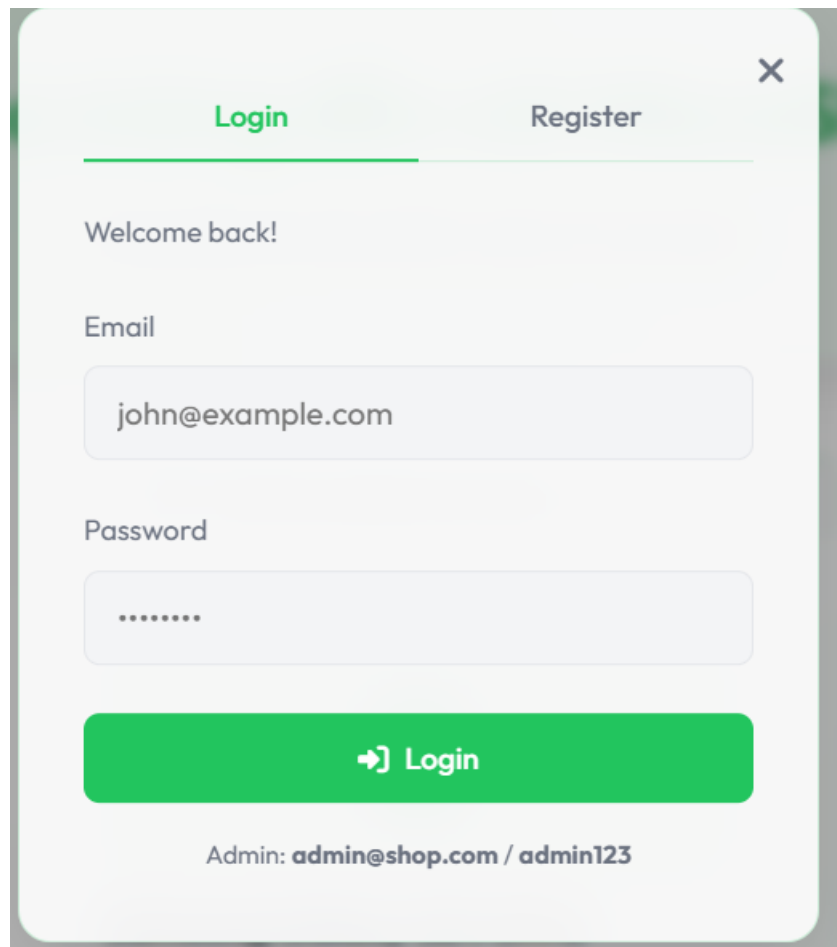
Cấu trúc này cũng phản ánh đúng nguyên tắc phân tách trách nhiệm trong thiết kế phần mềm. Mỗi service không chỉ là một thư mục riêng, mà còn biểu diễn một ranh giới nghiệp vụ riêng. Điều này giúp hạn chế sự phụ thuộc giữa các miền chức năng, đồng thời tạo tiền đề để hệ thống có thể mở rộng theo từng thành phần trong tương lai.

3.2. Kết quả triển khai giao diện đăng nhập và đăng ký

Một trong những chức năng đầu tiên của hệ thống là cho phép người dùng đăng ký và đăng nhập. Đây là nhóm chức năng cơ bản nhưng quan trọng, vì nó giúp hệ thống xác định người dùng trước khi thực hiện các thao tác liên quan đến giỏ hàng, đặt hàng

hoặc xem lịch sử mua hàng. Trong kiến trúc của hệ thống, nhóm chức năng này được xử lý bởi user-service.

Ở phía giao diện, người dùng nhập thông tin tài khoản như họ tên, email và mật khẩu khi đăng ký, hoặc nhập email và mật khẩu khi đăng nhập. Dữ liệu này được frontend gửi đến API Gateway, sau đó Gateway định tuyến request đến user-service. Service này xử lý dữ liệu đầu vào, kiểm tra thông tin người dùng và trả kết quả về cho frontend. Nếu thao tác thành công, frontend sẽ hiển thị trạng thái đăng nhập hoặc chuyển người dùng sang giao diện chính của hệ thống.



The image shows a modal window for user authentication. At the top, there are two tabs: 'Login' (highlighted in green) and 'Register'. A close button (X) is in the top right corner. Below the tabs, the text 'Welcome back!' is displayed. There are two input fields: 'Email' with the value 'john@example.com' and 'Password' with masked characters '.....'. A green button with a right arrow icon and the text 'Login' is positioned below the password field. At the bottom, the text 'Admin: admin@shop.com / admin123' is shown.

Hình 3.2: Giao diện đăng nhập của hệ thống

The image shows a registration form with a light gray background and rounded corners. At the top right is a close button (X). Below it are two tabs: 'Login' and 'Register', with 'Register' being the active tab. The text 'Join us to start shopping' is displayed. The form contains three input fields: 'Full Name' with the value 'John Doe', 'Email' with the value 'john@example.com', and 'Password' with masked characters '.....'. At the bottom is a large green button with a white user icon and the text 'Register & Login'.

Hình 3.3: Giao diện đăng ký của hệ thống

Hình 3.2 và 3.3 minh họa giao diện đăng nhập và đăng ký của hệ thống. Giao diện này là điểm bắt đầu của luồng sử dụng đối với người dùng. Từ góc nhìn người dùng cuối, đây chỉ là một biểu mẫu nhập thông tin tài khoản. Tuy nhiên, từ góc nhìn hệ thống phân tán, thao tác này thể hiện sự phối hợp giữa nhiều thành phần: frontend tiếp nhận dữ liệu, API Gateway định tuyến request, user-service xử lý nghiệp vụ và user-db lưu trữ hoặc truy vấn dữ liệu người dùng.

Chức năng đăng ký giúp tạo mới người dùng trong hệ thống. Khi người dùng nhập thông tin và gửi biểu mẫu, hệ thống cần kiểm tra dữ liệu đầu vào, lưu thông tin tài khoản và trả về kết quả đăng ký. Chức năng đăng nhập giúp xác thực thông tin tài khoản để người dùng có thể tiếp tục sử dụng các chức năng còn lại. Sau khi đăng nhập thành công, frontend có thể lưu thông tin người dùng ở phía client để phục vụ các thao tác tiếp theo như thêm sản phẩm vào giỏ hàng hoặc tạo đơn hàng.

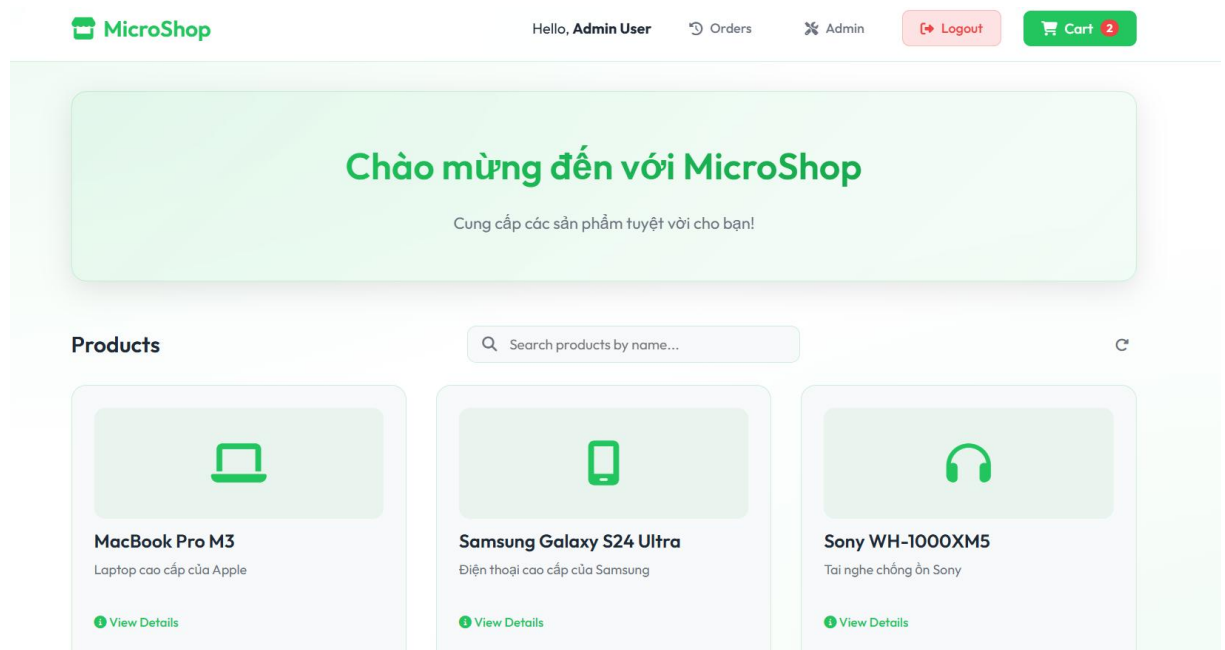
Về mặt thiết kế, chức năng này cho thấy sự phân tách giữa giao diện và xử lý nghiệp vụ. Frontend không tự xử lý nghiệp vụ người dùng mà chỉ gửi request đến hệ thống backend. user-service là nơi chịu trách nhiệm chính đối với dữ liệu người dùng. Điều này phù hợp với nguyên tắc phân chia miền nghiệp vụ trong microservices: mỗi service quản lý một nhóm chức năng và dữ liệu riêng.

Tuy nhiên, nhóm chức năng đăng nhập và đăng ký cũng là nơi cần chú ý nhiều về bảo mật. Trong một hệ thống thực tế, các thông tin nhạy cảm như mật khẩu cần được mã hóa hoặc băm trước khi lưu trữ. Ngoài ra, hệ thống nên sử dụng cơ chế xác thực như JWT hoặc session an toàn, đồng thời kiểm tra quyền truy cập ở backend thay vì chỉ dựa vào frontend. Đây là một trong những hướng cải tiến quan trọng sẽ được đề cập ở phần sau.

3.3. Kết quả triển khai giao diện danh sách sản phẩm

Sau khi truy cập hệ thống, người dùng có thể xem danh sách sản phẩm đang được cung cấp. Đây là một trong những chức năng trung tâm của hệ thống thương mại điện tử, vì sản phẩm là đối tượng chính mà người dùng quan tâm trước khi thêm vào giỏ hàng hoặc đặt hàng. Chức năng này được xử lý bởi product-service.

Khi người dùng mở trang sản phẩm, frontend gửi request đến API Gateway theo nhóm endpoint sản phẩm. Gateway định tuyến request đến product-service. Service này truy vấn cơ sở dữ liệu sản phẩm, lấy danh sách sản phẩm và trả kết quả về frontend. Sau đó, frontend hiển thị danh sách sản phẩm lên giao diện người dùng. Tùy theo giao diện được triển khai, mỗi sản phẩm có thể hiển thị các thông tin như tên sản phẩm, mô tả, giá bán, số lượng tồn kho và nút thêm vào giỏ hàng.



Hình 3.4: Giao diện danh sách sản phẩm của hệ thống

Hình 3.4 minh họa giao diện danh sách sản phẩm của hệ thống. Đây là giao diện giúp người dùng quan sát các sản phẩm hiện có và lựa chọn sản phẩm phù hợp. Về mặt chức năng, hình ảnh này cho thấy hệ thống đã triển khai được nghiệp vụ hiển thị sản phẩm. Về mặt kiến trúc, nó thể hiện luồng giao tiếp REST đồng bộ giữa frontend, API Gateway và product-service.

Chức năng danh sách sản phẩm có vai trò quan trọng trong việc kiểm chứng sự phân chia service theo miền nghiệp vụ. Thông tin sản phẩm không nằm trong user-service, cart-service hay order-service, mà được quản lý riêng bởi product-service. Điều này giúp dữ liệu sản phẩm có ranh giới sở hữu rõ ràng. Khi cần thêm, sửa, xóa hoặc tìm kiếm sản phẩm, các thao tác đều được xử lý thông qua service này.

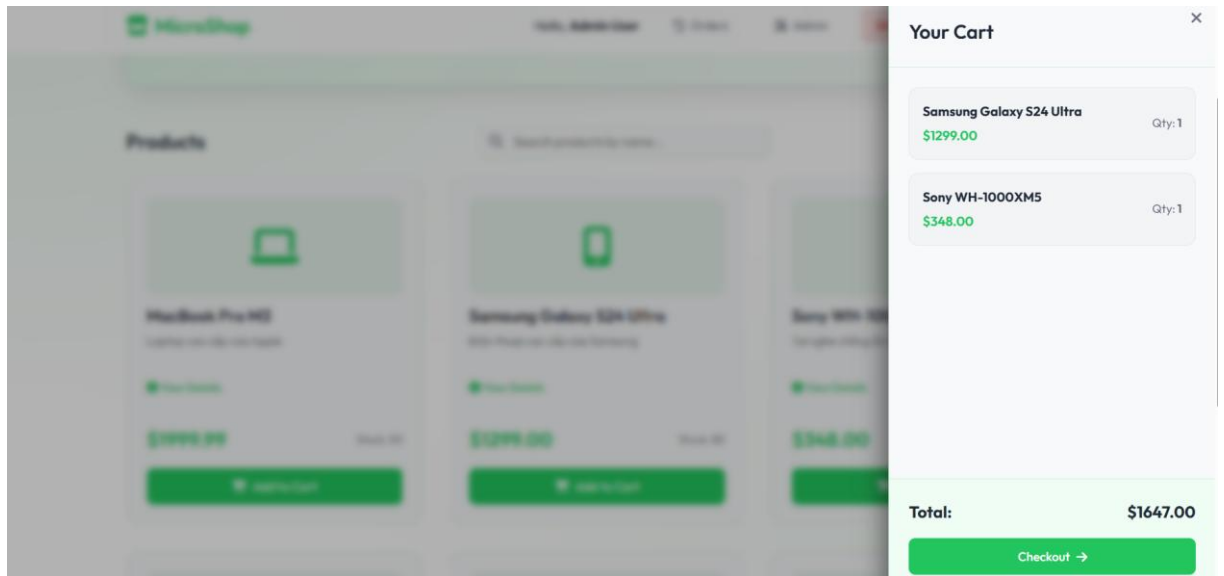
Việc triển khai danh sách sản phẩm cũng tạo tiền đề cho các luồng nghiệp vụ tiếp theo. Người dùng cần xem sản phẩm trước khi thêm vào giỏ hàng. Khi đặt hàng, order-service cũng cần truy vấn thông tin sản phẩm để tính tổng giá trị đơn hàng và kiểm tra dữ liệu cần thiết. Vì vậy, product-service không chỉ phục vụ giao diện hiển thị sản phẩm, mà còn tham gia vào luồng checkout ở phía backend.

Từ góc nhìn hệ thống phân tán, chức năng này tương đối đơn giản vì chủ yếu là một luồng đọc dữ liệu đồng bộ. Tuy nhiên, trong thực tế, đây lại có thể là một trong những chức năng chịu tải cao nhất của hệ thống thương mại điện tử. Nếu số lượng người dùng tăng lên, service sản phẩm có thể cần được tối ưu bằng cache, phân trang, tìm kiếm nâng cao hoặc nhân bản service để tăng khả năng đáp ứng.

3.4. Kết quả triển khai giao diện giỏ hàng

Giỏ hàng là thành phần trung gian giữa quá trình xem sản phẩm và quá trình đặt hàng. Sau khi người dùng chọn một sản phẩm, hệ thống cho phép thêm sản phẩm đó vào giỏ hàng với số lượng mong muốn. Dữ liệu giỏ hàng thường bao gồm mã người dùng, mã sản phẩm và số lượng sản phẩm. Trong hệ thống này, giỏ hàng được quản lý bởi cart-service.

Khi người dùng nhấn nút thêm sản phẩm vào giỏ hàng, frontend gửi request chứa các thông tin cần thiết đến API Gateway. Gateway định tuyến request đến cart-service. Service này kiểm tra dữ liệu giỏ hàng và lưu vào cơ sở dữ liệu riêng. Nếu sản phẩm đã tồn tại trong giỏ hàng của người dùng, hệ thống có thể cập nhật số lượng thay vì tạo một bản ghi mới. Cách xử lý này giúp dữ liệu giỏ hàng gọn hơn và phản ánh đúng hành vi mua hàng của người dùng.



Hình 3.5: Giao diện giỏ hàng của hệ thống

Hình 3.5 minh họa giao diện giỏ hàng của người dùng. Giao diện này cho phép người dùng kiểm tra các sản phẩm đã chọn trước khi tiến hành đặt hàng. Người dùng có thể xem danh sách sản phẩm trong giỏ, số lượng, thông tin liên quan và chuẩn bị xác nhận mua hàng.

Về mặt kiến trúc, giao diện giỏ hàng cho thấy vai trò riêng của cart-service trong hệ thống. Thay vì lưu dữ liệu giỏ hàng trong frontend hoặc gộp vào service sản phẩm, hệ thống tách giỏ hàng thành một service riêng. Điều này giúp nghiệp vụ giỏ hàng có thể phát triển độc lập, chẳng hạn bổ sung chức năng cập nhật số lượng, xóa từng sản phẩm khỏi giỏ, tính tạm tổng tiền hoặc lưu giỏ hàng lâu dài theo người dùng.

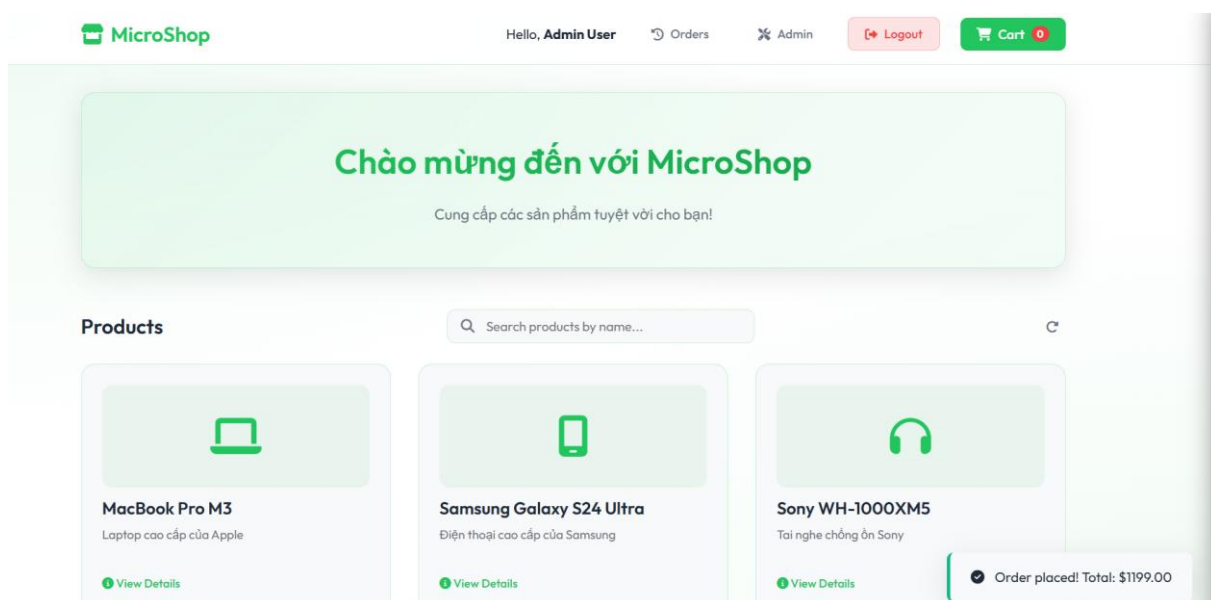
Chức năng giỏ hàng cũng là bước chuẩn bị trực tiếp cho luồng checkout. Khi người dùng xác nhận đặt hàng, order-service sẽ cần lấy dữ liệu giỏ hàng từ cart-service để biết người dùng muốn mua những sản phẩm nào. Do đó, dữ liệu giỏ hàng đóng vai trò là đầu vào quan trọng cho quá trình tạo đơn hàng.

Từ góc nhìn hệ thống phân tán, giỏ hàng là ví dụ cho việc dữ liệu được phân tán theo từng miền chức năng. cart-service sở hữu dữ liệu giỏ hàng, trong khi order-service không truy cập trực tiếp vào cơ sở dữ liệu giỏ hàng mà phải lấy dữ liệu thông qua API. Cách tổ chức này phù hợp với mô hình database-per-service, nhưng cũng làm phát sinh phụ thuộc giữa các service trong luồng checkout. Vì vậy, các cơ chế như timeout và circuit breaker có ý nghĩa quan trọng khi order-service gọi sang cart-service.

3.5. Kết quả triển khai luồng tạo đơn hàng

Luồng tạo đơn hàng là chức năng quan trọng nhất của hệ thống. Đây là luồng kết nối nhiều thành phần nhất và thể hiện rõ nhất đặc trưng của kiến trúc microservices. Khi người dùng xác nhận mua hàng, frontend gửi request tạo đơn hàng đến API Gateway. Gateway chuyển tiếp request đến order-service. Tại đây, order-service đóng vai trò điều phối: lấy dữ liệu giỏ hàng từ cart-service, lấy thông tin sản phẩm từ product-service, tính tổng giá trị đơn hàng, lưu đơn hàng và phát sinh sự kiện sau khi đơn hàng được tạo.

Khác với các chức năng như đăng nhập hoặc xem sản phẩm, luồng tạo đơn hàng không chỉ là thao tác đọc hoặc ghi dữ liệu đơn giản. Nó yêu cầu nhiều service phối hợp với nhau. cart-service cung cấp danh sách sản phẩm người dùng muốn mua. product-service cung cấp thông tin sản phẩm cần thiết. order-service tổng hợp dữ liệu, tạo đơn hàng và lưu vào cơ sở dữ liệu đơn hàng. Sau đó, RabbitMQ được sử dụng để truyền sự kiện cho các service khác xử lý tiếp.



Hình 3.6: Giao diện đặt đơn hàng thành công của hệ thống

Hình 3.6 minh họa kết quả tạo đơn hàng thành công. Hình này là giao diện frontend thông báo đặt hàng thành công, kết quả trả về sau khi gọi API hoặc thông tin đơn hàng được hiển thị sau checkout. Đây là bằng chứng trực quan cho thấy hệ thống đã hoàn thành phần xử lý chính của quy trình mua hàng.

Trong luồng này, order-service là service trung tâm nhưng không phải là nơi sở hữu toàn bộ dữ liệu. Service này chỉ sở hữu dữ liệu đơn hàng. Các dữ liệu khác như giỏ hàng và sản phẩm thuộc về service khác. Vì vậy, để tạo được đơn hàng, order-service phải giao tiếp với cart-service và product-service thông qua API. Đây là một đặc điểm

rất điển hình của hệ thống microservices: một nghiệp vụ tổng thể có thể cần sự phối hợp của nhiều service độc lập.

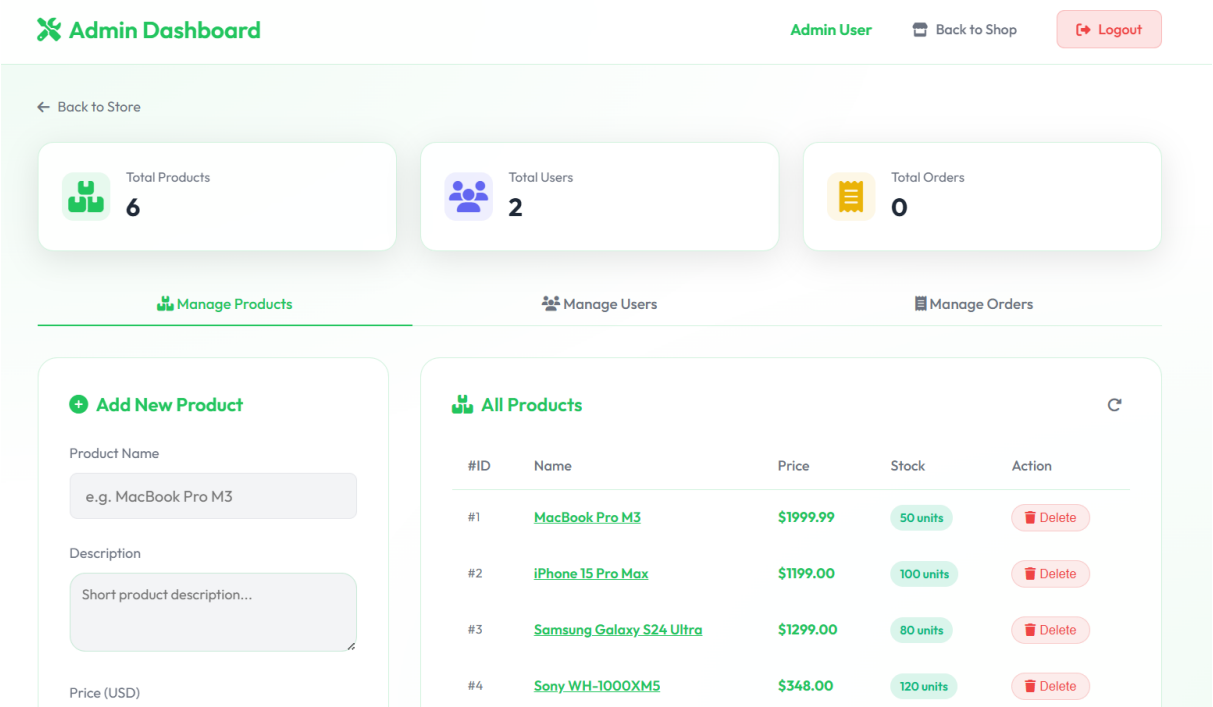
Luồng tạo đơn hàng cũng cho thấy sự đánh đổi của kiến trúc phân tán. Việc tách service giúp hệ thống rõ ràng hơn về trách nhiệm và dễ mở rộng hơn, nhưng đồng thời làm cho luồng nghiệp vụ phức tạp hơn so với hệ thống nguyên khối. Nếu cart-service hoặc product-service không phản hồi, quá trình tạo đơn hàng có thể bị ảnh hưởng. Do đó, hệ thống cần có các cơ chế bảo vệ như timeout, circuit breaker và xử lý lỗi phù hợp.

Về mặt kết quả, khi đơn hàng được tạo thành công, hệ thống đã hoàn tất pha đồng bộ của luồng checkout. Người dùng nhận được phản hồi rằng thao tác đặt hàng đã thành công. Tuy nhiên, các thao tác hậu xử lý như cập nhật tồn kho, cập nhật thông tin người dùng và xóa giỏ hàng được chuyển sang pha bất đồng bộ thông qua RabbitMQ. Nội dung này được minh họa ở phần tiếp theo.

3.6. Kết quả triển khai giao diện quản trị sản phẩm

Bên cạnh giao diện dành cho người dùng, hệ thống còn có giao diện quản trị sản phẩm. Đây là phần phục vụ người quản trị trong việc theo dõi và thay đổi dữ liệu sản phẩm. Trong một hệ thống thương mại điện tử, dữ liệu sản phẩm là một trong những nhóm dữ liệu quan trọng nhất, vì nó ảnh hưởng trực tiếp đến trải nghiệm mua hàng, tồn kho và đơn hàng.

Thông qua giao diện quản trị, quản trị viên có thể quan sát danh sách sản phẩm, thêm sản phẩm mới, chỉnh sửa thông tin sản phẩm hoặc xóa sản phẩm khỏi hệ thống. Các thao tác này được gửi từ frontend admin đến API Gateway, sau đó Gateway định tuyến đến product-service. Service này xử lý nghiệp vụ và cập nhật dữ liệu trong product-db.



Hình 3.7: Giao diện admin quản trị sản phẩm của hệ thống

Hình 3.7 minh họa giao diện admin quản lý sản phẩm. Hình ảnh này cho thấy hệ thống không chỉ phục vụ người dùng mua hàng, mà còn hỗ trợ một phần chức năng quản trị. Đây là yêu cầu cần thiết đối với một nền tảng thương mại điện tử, vì dữ liệu sản phẩm cần được cập nhật thường xuyên trong quá trình vận hành.

Về mặt kiến trúc, chức năng quản trị sản phẩm vẫn sử dụng lại product-service, thay vì xây dựng một service riêng biệt không cần thiết. Điều này giúp hệ thống giữ được sự nhất quán trong xử lý dữ liệu sản phẩm. Mọi thao tác liên quan đến sản phẩm đều đi qua cùng một service chịu trách nhiệm chính. Cách tổ chức này phù hợp với nguyên tắc một service sở hữu một miền nghiệp vụ.

Tuy vậy, giao diện quản trị cũng đặt ra yêu cầu cao hơn về bảo mật. Các thao tác như thêm, sửa, xóa sản phẩm không nên được cho phép với mọi người dùng. Trong một hệ thống hoàn chỉnh, backend cần kiểm tra quyền quản trị một cách chặt chẽ, thay vì chỉ dựa vào kiểm tra ở frontend. Đây là một điểm cần cải tiến quan trọng nếu hệ thống được phát triển theo hướng thực tế hơn.

3.7. Đánh giá kết quả đạt được

Từ các hình ảnh minh họa và kết quả triển khai, có thể thấy hệ thống đã đạt được các chức năng chính của một nền tảng thương mại điện tử ở mức đồ án. Người dùng có thể đăng ký hoặc đăng nhập, xem danh sách sản phẩm, thêm sản phẩm vào giỏ hàng và tạo đơn hàng. Bên cạnh đó, hệ thống cũng cung cấp giao diện quản trị để thao tác với dữ liệu sản phẩm.

Điểm đáng chú ý là các chức năng này không được triển khai trong một khối ứng dụng duy nhất. Hệ thống đã phân tách các miền nghiệp vụ thành nhiều service độc lập, đúng với định hướng microservices. Mỗi service có trách nhiệm riêng, có dữ liệu riêng và giao tiếp với các thành phần khác thông qua API hoặc message broker. Điều này giúp hệ thống thể hiện rõ các nội dung trọng tâm của môn Các hệ thống phân tán.

Về mặt giao tiếp, hệ thống kết hợp cả REST đồng bộ và RabbitMQ bất đồng bộ. Các thao tác cần phản hồi ngay cho người dùng, như đăng nhập, xem sản phẩm, thêm vào giỏ hàng và tạo đơn hàng, sử dụng REST API. Các tác vụ sau khi tạo đơn, như cập nhật tồn kho, cập nhật thông tin người dùng và xóa giỏ hàng, được xử lý thông qua RabbitMQ. Cách kết hợp này cho thấy hệ thống đã áp dụng được mô hình event-driven và eventual consistency trong một nghiệp vụ cụ thể.

Về mặt triển khai, hệ thống có thể được đóng gói và vận hành theo mô hình nhiều thành phần. Điều này giúp việc trình bày đồ án trở nên rõ ràng hơn, vì người đọc có thể

thấy hệ thống gồm nhiều service phối hợp với nhau chứ không chỉ là một website đơn giản. Cây thư mục dự án, giao diện người dùng, giao diện quản trị và kết quả xử lý event là các minh chứng trực quan cho điều này.

Nhìn chung, kết quả triển khai cho thấy hệ thống đã đáp ứng được mục tiêu cơ bản của đề tài: xây dựng một hệ thống thương mại điện tử theo kiến trúc microservices, có giao diện người dùng, giao diện quản trị, các service nghiệp vụ riêng và cơ chế xử lý sự kiện sau khi tạo đơn hàng.

3.8. Hạn chế của hệ thống

Mặc dù hệ thống đã triển khai được nhiều chức năng quan trọng, vẫn còn một số hạn chế cần tiếp tục hoàn thiện. Hạn chế đầu tiên là bảo mật. Hệ thống hiện mới tập trung vào chức năng và kiến trúc, trong khi các cơ chế bảo vệ như mã hóa mật khẩu, xác thực bằng token, phân quyền ở backend và bảo vệ các API quản trị cần được bổ sung đầy đủ hơn. Đặc biệt, với giao diện admin, việc kiểm tra quyền ở phía backend là rất quan trọng để tránh trường hợp người dùng không có quyền vẫn có thể gọi API quản trị.

Hạn chế thứ hai là cơ chế xử lý sự kiện chưa hoàn chỉnh ở mức sản phẩm thực tế. Hệ thống đã sử dụng RabbitMQ để truyền sự kiện sau khi tạo đơn hàng, nhưng cần tiếp tục bổ sung cơ chế idempotency để đảm bảo một message nếu bị xử lý lại nhiều lần cũng không làm sai lệch dữ liệu. Đây là vấn đề quan trọng trong hệ thống event-driven, đặc biệt với các thao tác như trừ tồn kho hoặc tăng số đơn hàng của người dùng.

Hạn chế thứ ba là khả năng quan sát hệ thống còn có thể cải thiện. Trong hệ thống nhiều service, việc chỉ xem log riêng lẻ của từng service có thể gây khó khăn khi cần truy vết một lỗi đi qua nhiều thành phần. Hệ thống nên được bổ sung log tập trung, metrics và distributed tracing để dễ theo dõi luồng request, độ trễ, lỗi service và trạng thái message trong RabbitMQ.

3.9. Hướng cải tiến và phát triển

Từ các hạn chế đã nêu, hệ thống có thể được cải tiến theo nhiều hướng. Trước hết, cần hoàn thiện bảo mật. Hệ thống nên bổ sung Spring Security, xác thực bằng JWT hoặc session an toàn, băm mật khẩu trước khi lưu trữ, phân quyền theo vai trò ở backend và bảo vệ các API quản trị. Đây là yêu cầu quan trọng nếu hệ thống được phát triển theo hướng gần với sản phẩm thực tế.

Thứ hai, cần nâng cao độ tin cậy của cơ chế xử lý sự kiện. Các consumer nên được thiết kế theo hướng idempotent, có event id hoặc bảng ghi nhận message đã xử lý để tránh cập nhật trùng lặp. Ngoài ra, có thể bổ sung transactional outbox để đảm bảo thao tác lưu đơn hàng và phát sự kiện được phối hợp tin cậy hơn. Với các nghiệp vụ phức tạp hơn, hệ thống có thể nghiên cứu Saga pattern để quản lý các giao dịch trải dài qua nhiều service.

Thứ ba, cần cải thiện khả năng quan sát và giám sát. Hệ thống có thể tích hợp Prometheus và Grafana để theo dõi metrics, kết hợp với Zipkin, Jaeger hoặc Grafana Tempo để truy vết request liên service. RabbitMQ cũng nên được giám sát thường xuyên để phát hiện message tồn đọng hoặc message bị chuyển vào DLQ.

Thứ tư, hệ thống có thể cải thiện hiệu năng bằng cách tối ưu luồng checkout. Nếu giỏ hàng có nhiều sản phẩm, việc order-service gọi product-service nhiều lần có thể làm tăng độ trễ. Một hướng cải tiến là bổ sung API lấy danh sách sản phẩm theo nhiều productId trong một request, từ đó giảm số lượng remote call. Ngoài ra, có thể bổ sung cache cho dữ liệu sản phẩm ít thay đổi, nhưng cần đảm bảo cache không làm sai lệch thông tin tồn kho trong các nghiệp vụ quan trọng.

Kết luận chương

Chương 3 đã trình bày kết quả triển khai hệ thống thông qua các hình ảnh minh họa chính, bao gồm cây thư mục dự án, giao diện đăng nhập hoặc đăng ký, danh sách sản phẩm, giỏ hàng, kết quả tạo đơn hàng, xử lý sự kiện sau khi tạo đơn và giao diện quản trị sản phẩm. Những hình ảnh này cho thấy hệ thống đã được triển khai thành một ứng dụng thương mại điện tử có các chức năng cơ bản cho người dùng và quản trị viên.

Bên cạnh việc đáp ứng chức năng nghiệp vụ, hệ thống còn thể hiện được các đặc trưng quan trọng của kiến trúc microservices và hệ thống phân tán. Các service được tách theo miền chức năng, dữ liệu được tổ chức riêng cho từng service, request được định tuyến thông qua API Gateway và các tác vụ hậu xử lý sau khi tạo đơn hàng được thực hiện bất đồng bộ thông qua RabbitMQ. Điều này cho thấy hệ thống không chỉ là một website bán hàng thông thường, mà còn là một mô hình ứng dụng phân tán ở quy mô nhỏ.

Tuy nhiên, hệ thống vẫn cần tiếp tục hoàn thiện ở nhiều khía cạnh như bảo mật, xử lý trùng lặp sự kiện, giám sát tập trung, lưu trữ bền vững, kiểm thử hiệu năng và triển khai ở môi trường có khả năng mở rộng cao hơn.

TỔNG KẾT

Sau quá trình nghiên cứu, phân tích, thiết kế và triển khai, nhóm đã hoàn thành đề tài “Phát triển hệ thống thương mại điện tử theo kiến trúc Microservices” trong khuôn khổ môn học Các hệ thống phân tán. Đề tài đã vận dụng các kiến thức lý thuyết về hệ thống phân tán vào một bài toán phần mềm cụ thể, gần gũi với thực tế là xây dựng một nền tảng thương mại điện tử gồm nhiều thành phần độc lập phối hợp với nhau.

Về mặt lý thuyết, báo cáo đã trình bày các khái niệm nền tảng liên quan đến hệ thống phân tán và kiến trúc microservices, bao gồm tính trong suốt, khả năng mở rộng, khả năng chịu lỗi, API Gateway, giao tiếp REST đồng bộ, message broker, RabbitMQ, database-per-service, retry, dead-letter queue, kiến trúc hướng sự kiện, eventual consistency, tracing, observability và Docker Compose. Đây là cơ sở quan trọng để hiểu cách các thành phần trong hệ thống được tổ chức, giao tiếp và xử lý dữ liệu trong môi trường phân tán.

Về mặt thiết kế, hệ thống đã được phân tích theo hướng chia tách các miền nghiệp vụ thành nhiều service độc lập. Các chức năng chính như quản lý người dùng, quản lý sản phẩm, quản lý giỏ hàng và xử lý đơn hàng được tổ chức thành các service riêng. API Gateway được sử dụng làm điểm vào thống nhất cho phía frontend, giúp che giấu cấu trúc nội bộ của hệ thống. Mỗi service có cơ sở dữ liệu riêng, phù hợp với mô hình database-per-service. Bên cạnh đó, RabbitMQ được sử dụng để truyền sự kiện sau khi

đơn hàng được tạo, giúp tách các tác vụ hậu xử lý khỏi luồng request chính và thể hiện mô hình event-driven trong hệ thống microservices.

Về mặt triển khai, hệ thống đã xây dựng được các chức năng cơ bản của một nền tảng thương mại điện tử. Người dùng có thể đăng ký, đăng nhập, xem danh sách sản phẩm, thêm sản phẩm vào giỏ hàng và thực hiện đặt hàng. Hệ thống cũng có giao diện quản trị để hỗ trợ quản lý sản phẩm. Luồng checkout là luồng nghiệp vụ quan trọng nhất, vì nó thể hiện sự phối hợp giữa nhiều service: order-service lấy dữ liệu từ cart-service và product-service, lưu đơn hàng vào cơ sở dữ liệu riêng, sau đó phát sự kiện để các service khác tiếp tục cập nhật tồn kho, thông tin người dùng và giỏ hàng.

Kết quả đạt được cho thấy hệ thống không chỉ là một website bán hàng thông thường, mà là một mô hình ứng dụng phân tán ở quy mô nhỏ. Hệ thống thể hiện rõ các đặc trưng quan trọng của microservices như phân tách service theo miền nghiệp vụ, tách cơ sở dữ liệu theo service, giao tiếp qua API Gateway, kết hợp REST đồng bộ với RabbitMQ bất đồng bộ và chấp nhận tính nhất quán cuối cùng trong các nghiệp vụ trải dài qua nhiều service. Các cơ chế như retry, dead-letter queue và circuit breaker cũng cho thấy hệ thống đã bước đầu quan tâm đến khả năng chịu lỗi trong môi trường phân tán.

Tuy nhiên, do phạm vi đề án còn giới hạn, hệ thống vẫn còn một số điểm cần tiếp tục hoàn thiện. Về bảo mật, hệ thống cần bổ sung xác thực và phân quyền ở phía backend, mã hóa mật khẩu, bảo vệ các API quản trị và quản lý thông tin người dùng an toàn hơn. Về độ tin cậy dữ liệu, hệ thống cần bổ sung cơ chế idempotency, event id hoặc transactional outbox để hạn chế nguy cơ xử lý trùng lặp hoặc mất sự kiện. Về khả năng quan sát, hệ thống nên được tích hợp thêm log tập trung, metrics và distributed tracing để hỗ trợ theo dõi, phân tích lỗi và đánh giá hiệu năng. Ngoài ra, nếu muốn triển khai trong môi trường thực tế, hệ thống cần có cơ chế healthcheck đầy đủ, lưu trữ bền vững, backup dữ liệu, quản lý secret và khả năng mở rộng bằng các nền tảng orchestration như Kubernetes.

Thông qua đề tài này, nhóm đã có cơ hội củng cố kiến thức về hệ thống phân tán, đồng thời hiểu rõ hơn những lợi ích và thách thức khi áp dụng kiến trúc microservices vào một bài toán thực tế. Kiến trúc microservices giúp hệ thống linh hoạt, dễ mở rộng và dễ bảo trì hơn, nhưng cũng làm phát sinh nhiều vấn đề mới như giao tiếp liên service, đồng bộ dữ liệu, xử lý lỗi, bảo mật và giám sát hệ thống. Đây là những nội dung quan trọng mà nhóm cần tiếp tục nghiên cứu và hoàn thiện trong các phiên bản phát triển tiếp theo.

Nhìn chung, đề tài đã đạt được mục tiêu đặt ra ban đầu: xây dựng và phân tích một hệ thống thương mại điện tử theo kiến trúc microservices, có sự kết hợp giữa giao tiếp đồng bộ và bất đồng bộ, có tách dữ liệu theo từng service và có khả năng minh họa các đặc trưng cơ bản của hệ thống phân tán. Dù vẫn còn những hạn chế nhất định, hệ thống là nền tảng tốt để tiếp tục mở rộng theo hướng hoàn thiện bảo mật, nâng cao độ tin cậy, cải thiện khả năng quan sát, tối ưu hiệu năng và tiến gần hơn đến một hệ thống thương mại điện tử có khả năng vận hành trong môi trường thực tế.

TÀI LIỆU THAM KHẢO

- [1] T. T. M. Ngọc, “Microservices và API Gateway: Hiểu rõ khác biệt và tầm quan trọng,” Sunteco, Jul. 27, 2023. [Online]. Available: <https://sunteco.vn/microservices-va-api-gateway-hieu-ro-khac-biet-va-tam-quan-trong/>. [Accessed: May 27, 2026].
- [2] C. Richardson, “Pattern: Database per service,” Microservices.io. Accessed: May 27, 2026. [Online]. Available: <https://microservices.io/patterns/data/database-per-service.html>
- [3] Microsoft, “Retry pattern,” Microsoft Learn, Azure Architecture Center. Accessed: May 27, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/patterns/retry>
- [4] C. Richardson, “Pattern: API Gateway / Backends for Frontends,” Microservices.io. Accessed: May 27, 2026. [Online]. Available: <https://microservices.io/patterns/apigateway.html>

- [5] M. Ozkaya, “The Database-per-Service Pattern,” Medium, Sep. 6, 2021. [Online]. Available: <https://medium.com/design-microservices-architecture-with-patterns/the-database-per-service-pattern-9d511b882425>. [Accessed: May 27, 2026].
- [6] Microsoft, “Communication in a microservice architecture,” Microsoft Learn. Accessed: May 27, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/architect-microservice-container-applications/communication-in-microservice-architecture>
- [7] RabbitMQ, “RabbitMQ Tutorials,” RabbitMQ Documentation. Accessed: May 27, 2026. [Online]. Available: <https://www.rabbitmq.com/tutorials>
- [8] N. N. Vãn, “Tìm hiểu về RabbitMQ,” Viblo, Oct. 6, 2019. [Online]. Available: <https://viblo.asia/p/tim-hieu-ve-rabbitmq-OeVKB8bMlkW>. [Accessed: May 27, 2026].
- [9] Microsoft, “Event-driven architecture style,” Microsoft Learn, Azure Architecture Center. Accessed: May 27, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>
- [10] OpenTelemetry, “Signals,” OpenTelemetry Documentation. Accessed: May 27, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/>
- [11] Google Cloud, “About Google Cloud Observability,” Google Cloud Documentation. Accessed: May 27, 2026. [Online]. Available: <https://cloud.google.com/stackdriver/docs>